Adaptive Wind Power Forecasting using Concept Drift Detection

A PROJECT REPORT

Submitted by

THANYA B [211423247227]

THEJESSRI S [211423247229]

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "Adaptive Wind Power Forecasting using Concept Drift Detection" is the bonafide work of **THANYA B (211423247227)** and **THEJESSRI S (211423247229)**, who carried out the project work under my supervision.

Certified that the above candidates were examined in the End Semester Project Viva-Voce Examination held on .....

SIGNATURE

Dr. G. Suresh M.Tech., Ph.D.
Head of the Department,
Professor,
Department of Artificial Intelligence
and Machine Learning,
Panimalar Engineering College
Chennai – 600 123

SIGNATURE

Mr. M. Ezhilvendhan
Supervisor,
Department of Artificial Intelligence
and Machine Learning,
Panimalar Engineering College
Chennai – 600 123

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION BY THE STUDENTS

WE, **THANYA B (211423247227)** AND **THEJESSRI S (211423247229)**, HEREBY DECLARE THAT THIS PROJECT REPORT TITLED “ADAPTIVE WIND POWER FORECASTING USING CONCEPT DRIFT DETECTION”, UNDER THE GUIDANCE OF **MR. M. EZHILVENDHAN**, IS THE ORIGINAL WORK DONE BY US, AND WE HAVE NOT PLAGIARIZED OR SUBMITTED IT TO ANY OTHER DEGREE IN ANY UNIVERSITY BY US.

THANYA B (211423247227)

THEJESSRI S (211423247229)

ACKNOWLEDGEMENT

We express our deep gratitude to our respected Secretary and Correspondent, **Dr. P. CHINNADURAI, M.A., Ph.D.**, for his kind words and enthusiastic motivation, which inspired us a lot in completing this project.

We would like to extend our heartfelt thanks to our Directors, **Tmt. C. VIJAYARAJESWARI**, **Dr. C. SAKTHIKUMAR, M.E., Ph.D.** and **Tmt. SARANYASREE SAKTHIKUMAR, B.E., M.B.A.**, for providing us with the necessary facilities for the completion of this project.

We also express our gratitude to our Principal, **Dr. K. MANI, M.E., Ph.D.**, for his timely concern and encouragement provided to us throughout the course.

We thank our Head of the Department of Artificial Intelligence and Machine Learning, **Dr. G. SURESH, M.Tech., Ph.D.**, Professor, for the support extended throughout the project. His encouragement and timely suggestions have greatly contributed to the successful completion of our project.

We would also like to express our sincere gratitude to our Project Coordinator, **Dr. B. PRABHA, M.Tech., Ph.D.**, Assistant Professor Grade 1, for her valuable guidance, continuous support and coordination throughout the project.

We would like to thank our Supervisor, **Mr. M. Ezhilvendhan**, and all the faculty members of the Department of Artificial Intelligence and Machine Learning for their advice and suggestions for the successful completion of the project.

THANYA B (211423247227)

THEJESSRI S (211423247229)

ABSTRACT

Wind-power forecasting models are usually trained once on historical data and then used unchanged. When weather regimes shift or turbines age, are derated or are affected by new wake effects, the relationship between weather and power changes (concept drift) and forecast accuracy deteriorates silently. Blind periodic retraining wastes computation when nothing has changed and still reacts late to an abrupt shift. This project develops and evaluates a lightweight, interpretable drift-adaptive forecasting framework that detects such changes and retrains the model only when the evidence justifies it.

The framework forecasts one-hour-ahead wind-farm power with a histogram gradient-boosted regressor using power history, wind speed, cyclically encoded wind direction and temperature. Prediction errors are standardised by forecast level and monitored with a Page-Hinkley test, while daily Kolmogorov-Smirnov and population-stability-index checks watch the input distribution. When an alarm passes a 48-hour cool-down, an adaptation controller retrains the model on a four-week window with recency and drift-aware sample weights. The approach is evaluated in a prequential (test-then-train) protocol on a controlled synthetic generator with a known abrupt drift at week 28, against a static model and a weekly-retrained model, over ten random seeds and a no-drift control scenario.

Under drift, the adaptive model reduced the mean absolute error from 134.7 kW to 33.7 kW (75.0% lower; nRMSE 8.92% to 2.65%). It detected the drift in all ten runs with a mean delay of 21.5 hours and no false alarm before the onset. It matched the accuracy of weekly retraining (35.3 kW, a small but statistically significant advantage for the adaptive model) with 4.9 times fewer retrainings. Without drift, the adaptive model raised no alarm and retrained never, whereas weekly retraining worsened the error by 30.2%. Sensitivity studies of the detector, cool-down, retraining window and drift magnitude show when adaptation pays off and when it does not.

The results are obtained on synthetic data and establish the behaviour of the method under controlled conditions; validation on measured multi-season wind-farm data is the main direction for future work. Overall, the framework offers an evidence-based, low-cost way to keep wind-power forecasts reliable under changing conditions and to quantify what the adaptation costs.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	Abstract	**v**
	Table of Contents	**vi**
	List of Figures	**ix**
	List of Tables	**xi**
	List of Abbreviations	**xii**
1	**Introduction**	**1**
1.1	Background of Wind-Power Forecasting	1
1.2	Concept Drift in Wind-Power Forecasting	2
1.3	Problem Statement	3
1.4	Motivation of the Project	4
1.5	Research Question and Hypotheses	4
1.6	Objectives of the Project	5
1.7	Scope and Limitations	5
1.8	Contributions	5
1.9	Organisation of the Report	6
2	**Literature Survey**	**7**
2.1	Approaches to Wind-Power Forecasting	7
2.2	Concept Drift: Theory and Detection	8
2.3	Drift-Aware Wind and Power Forecasting	8
2.4	Hybrid, Transfer and Domain-Adaptation Approaches	9
2.5	Summary of Surveyed Works	10
2.6	Research Gap and Positioning of This Work	11
3	**Theoretical Background**	**12**
3.1	Wind-Power Conversion	12
3.2	Forecasting Problem Formulation	12
3.3	Gradient-Boosted Regression Trees	13
3.4	Formal Definition of Concept Drift	13
3.5	Drift-Detection Methods	14
3.6	Drift-Aware Retraining	15
3.7	Evaluation Metrics	16
3.8	Prequential (Test-Then-Train) Evaluation	16
3.9	Statistical Tests for Comparison	16
4	**System Analysis and Design**	**17**
4.1	Existing System Overview	17
4.2	Proposed System Overview	17

CHAPTER NO.	TITLE	PAGE NO.
4.3	Advantages of the Proposed System	18
4.4	System Architecture	18
4.5	Data Flow Diagram	20
4.6	UML Design	21
4.7	Implementation Environment	22
4.8	Project Schedule	23
5	**Methodology and Implementation**	**24**
5.1	Data Description and Synthetic Generator	24
5.2	Preprocessing and Feature Engineering	28
5.3	Chronological Split and Streaming Protocol	29
5.4	Base Forecasting Model	30
5.5	Error Standardisation and Calibration	31
5.6	Drift Detectors	32
5.7	Adaptation Controller	32
5.8	Baseline Models	33
5.9	Module-Wise Implementation	33
5.10	Experimental Protocol	34
6	**Results and Discussion**	**36**
6.1	Overview of the Experiments	36
6.2	Forecast Accuracy under Concept Drift	36
6.3	Forecast Accuracy without Drift	41
6.4	Drift-Detection Performance	42
6.5	Computational Cost	45
6.6	Statistical Significance	46
6.7	Comparison of Drift Detectors	46
6.8	Sensitivity to the Detection Threshold	47
6.9	Sensitivity to Cool-Down and Retraining Window	48
6.10	Sensitivity to Drift Magnitude	50
6.11	Verification of the Hypotheses	51
6.12	Practical Guidance: When Does Adaptation Pay Off?	51
6.13	Limitations and Threats to Validity	51
7	**Conclusion and Future Scope**	**53**
7.1	Conclusion	53
7.2	Future Scope	54
A	**Sustainable Development Goals**	**55**
A.1	SDG 7: Affordable and Clean Energy	55
A.2	SDG 13: Climate Action	55

CHAPTER NO.	TITLE	PAGE NO.
A.3	SDG 9: Industry, Innovation and Infrastructure	55
A.4	SDG 12: Responsible Consumption and Production	55
A.5	SDG 4: Quality Education	55
B	**Source Code**	**56**
B.1	Simulation Engine (sim.py)	56
B.2	Experiment Driver for the Main Results	61
B.3	Experiment Driver for the Sensitivity Studies	62
C	**Output Screenshots**	**64**
D	**Per-Seed Results**	**66**
D.1	Configuration Summary	67
	References	**68**

LIST OF FIGURES

FIGURE NO.	TITLE OF THE FIGURE	PAGE NO.
1.1	Idealised 2 MW turbine power curve showing the four operating regions	2
1.2	Four common temporal patterns of concept drift	3
1.3	Original concept, virtual drift and real drift	3
2.1	Distribution of the twelve surveyed works by theme	11
3.1	Distribution comparison and the KS statistic as the largest gap between two ECDFs	15
3.2	Page-Hinkley test on a synthetic stream with a mean shift at step 150	15
4.1	Architecture of the drift-adaptive wind-power forecasting system	19
4.2	Flowchart of the online forecasting and adaptation loop	20
4.3	Level-1 data flow diagram	21
4.4	Use-case diagram	21
4.5	Sequence diagram of one forecasting and adaptation cycle	22
4.6	Class diagram of the main software components	22
4.7	Project schedule over eighteen weeks	23
5.1	First three weeks of the synthetic hourly stream (seed 0)	25
5.2	Weekly mean wind speed and power over the 106-week horizon with injected drift	25
5.3	Measured wind speed versus power before and after the injected drift	26
5.4	Wind-direction rose; the wake-affected sector is shown in red	27
5.5	Wind-speed distribution of the initial window against weeks 29-36, with and without injected drift	28
5.6	Raw angle versus sine/cosine encoding of wind direction	29
5.7	Correlation matrix of features and target in the initial window	29
5.8	Chronological split, online stream and drift onset	30
5.9	Permutation importance of the base model features	31
5.10	Mean absolute error per forecast-level bin (seed 0), used to standardise residuals	31
5.11	Sample weights used when retraining, with and without drift-aware down-weighting	33
6.1	MAE and RMSE of the three models in both scenarios (error bars: one standard deviation)	37
6.2	Weekly MAE of the three models in the drift scenario (seed 0)	38
6.3	Cumulative absolute forecast error in the drift scenario (seed 0)	38
6.4	Actual and forecast power around the drift onset (seed 0)	39
6.5	Predicted versus actual power after the drift transient (seed 0)	39

FIGURE NO.	TITLE OF THE FIGURE	PAGE NO.
6.6	Residual distributions after the drift transient (seed 0)	40
6.7	MAE by wind-speed bin before and after the drift (seed 0)	40
6.8	Weekly MAE of the three models in the no-drift scenario (seed 0)	41
6.9	Distribution of the MAE over the ten seeds in both scenarios	42
6.10	First-alarm detection delay for each random seed	43
6.11	Standardised error and Page-Hinkley statistic over the stream (seed 0)	43
6.12	Daily KS p-value and PSI of wind speed (seed 0, drift scenario)	44
6.13	Retraining events of the adaptive model for all ten seeds in both scenarios	45
6.14	Number of retrainings and cumulative retraining time	45
6.15	Comparison of drift detectors on accuracy, delay, retraining count and false alarms	47
6.16	Effect of the threshold λ on delay, retraining count and accuracy	48
6.17	Effect of the cool-down period on retraining count, accuracy and compute	49
6.18	Effect of the retraining window length	49
6.19	Static and adaptive MAE and the relative gain as a function of drift magnitude	50
C.1	Console output: synthetic data generation (seed 0)	64
C.2	Console output: base model training and calibration (seed 0)	64
C.3	Console output: online stream with drift-triggered retraining (seed 0)	64
C.4	Console output: evaluation summary over ten seeds	65
C.5	Console output: sensitivity summary (five seeds)	65

LIST OF TABLES

TABLE NO.	TITLE OF THE TABLE	PAGE NO.
1.1	Typical wind-power forecast horizons and their uses	1
1.2	Common causes of concept drift in wind-power data	2
1.3	Research hypotheses	4
1.4	Organisation of the report	6
2.1	Comparison of forecasting families	7
2.2	Families of concept-drift detectors	8
2.3	Summary of surveyed works (1 of 2)	10
2.4	Summary of surveyed works (2 of 2)	10
2.5	Capability comparison with surveyed approaches	11
3.1	Drift types and wind-power examples	13
4.1	Disadvantages of the existing systems	17
4.2	Functional modules of the proposed system	19
4.3	Software environment	22
5.1	Parameters of the synthetic data generator	27
5.2	Feature set used by the forecaster	28
5.3	Hyper-parameters of the forecaster	30
5.4	Models compared in every scenario	33
5.5	Experimental protocol	34
6.1	Forecast accuracy in the drift scenario (mean ± std over 10 seeds)	36
6.2	MAE by wind-speed bin (kW, seed 0)	40
6.3	Forecast accuracy in the no-drift scenario (mean ± std over 10 seeds)	41
6.4	Detection delay and early retraining activity per seed	42
6.5	Computational cost per run (mean over 10 seeds)	45
6.6	Paired comparison of MAE over ten seeds (drift scenario)	46
6.7	Comparison of drift detectors (mean over 5 seeds)	46
6.8	Effect of the Page-Hinkley threshold λ (mean over 5 seeds)	47
6.9	Effect of the cool-down period (drift scenario, 5 seeds)	48
6.10	Effect of the retraining window (drift scenario, 5 seeds)	49
6.11	Effect of drift magnitude (5 seeds)	50
6.12	Outcome of the research hypotheses	51
6.13	Guidance derived from the experiments	51
D.1	Per-seed accuracy in the drift scenario (kW)	66
D.2	Per-seed accuracy in the no-drift scenario (kW)	66
D.3	Final configuration of the adaptive system	67

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
ADWIN	Adaptive Windowing
AR(1)	First-order autoregressive process
ARIMA	Autoregressive integrated moving average
CRPS	Continuous ranked probability score
CUSUM	Cumulative sum control chart
DFD	Data flow diagram
ECDF	Empirical cumulative distribution function
GBM	Gradient boosting machine
GNN	Graph neural network
KS	Kolmogorov-Smirnov
LSTM	Long short-term memory
MAE	Mean absolute error
ML	Machine learning
NWP	Numerical weather prediction
nRMSE	Normalised root mean squared error
PH	Page-Hinkley test
PSI	Population stability index
RMSE	Root mean squared error
SCADA	Supervisory control and data acquisition
SDG	Sustainable Development Goal
TCN	Temporal convolutional network

CHAPTER 1
INTRODUCTION

1.1 Background of Wind-Power Forecasting

Wind energy has become one of the most important renewable sources of electricity. Unlike conventional thermal plants, whose output is dispatched on demand, a wind farm produces power only when the wind blows, and the amount produced depends on the instantaneous wind speed, wind direction, air temperature and the condition of the turbines. The resulting output is intermittent and variable on time scales ranging from seconds to seasons. For a power-system operator, this variability must be balanced second by second against demand, which makes an accurate forecast of wind generation a basic operational requirement.

Wind-power forecasts support several decisions. Grid operators use them to schedule reserves and to maintain frequency and voltage stability. Plant owners use them to bid in electricity markets and to avoid imbalance penalties. Maintenance planners use them to find low-wind windows for turbine servicing. Because the economic value of a forecast falls quickly as its error grows, forecasting research has concentrated on reducing error for every horizon of interest. Table 1.1 summarises the usual forecast horizons and the decisions that depend on them.

Table 1.1 Typical wind-power forecast horizons and their uses

Horizon class	Typical range	Main uses
Very short term	Seconds to 30 minutes	Turbine control, regulation reserve, ramp alerts
Short term	30 minutes to 6 hours	Economic dispatch, intraday trading, load following
Medium term	6 to 48 hours	Unit commitment, day-ahead market bids
Long term	Days to weeks	Maintenance scheduling, resource planning

This project works at the short-term end of the range: the model forecasts the wind-farm power of the next hour from the weather inputs available for that hour and from the recent power history. This is the horizon at which statistical and machine-learning models are normally preferred over physical weather models.

1.1.1 Why wind power is hard to forecast

Below the rated wind speed, the kinetic power available to a turbine rises with the cube of the wind speed. A small error in wind speed is therefore amplified into a large error in power: a 10% error in wind speed corresponds to about $(1.10)^3 - 1 \approx 33\%$ error in available power. Figure 1.1 shows the idealised power curve of a 2 MW turbine, with its

four operating regions. The steep, non-linear second region is where most forecast error is generated.

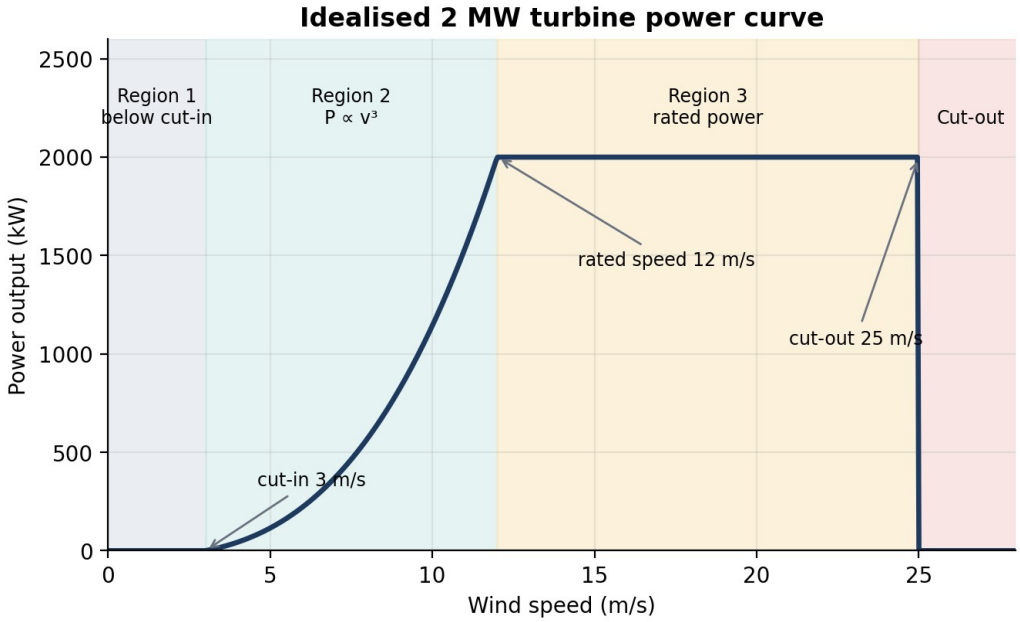


Figure 1.1 Idealised 2 MW turbine power curve showing the four operating regions

1.2 Concept Drift in Wind-Power Forecasting

Most forecasting models are trained once on historical data and then used unchanged. This works only if the relationship between the inputs (wind speed, direction, temperature, recent power) and the output (power) stays the same after training. This assumption, known as stationarity, rarely holds in practice. When the statistical relationship between the inputs and the target changes over time, the phenomenon is called **concept drift**. A model that was accurate when it was trained then becomes progressively less reliable, often without any visible warning.

In wind power, concept drift has several physical and operational causes. They are listed in Table 1.2 together with the way each one appears in the data.

Table 1.2 Common causes of concept drift in wind-power data

Cause	Mechanism	Drift type	Pattern in the data
Seasonal regime change	Different prevailing winds, air density and stability between seasons	Gradual / recurring	Distribution of speed and direction shifts slowly
Climate variability	Multi-year changes in mean wind resource	Incremental	Slow trend in mean wind speed
Turbine ageing or derating	Blade erosion, gearbox wear or imposed power limits lower output	Real, sudden or incremental	Same wind gives less power
Curtailment	Operator limits output for grid reasons	Real, intermittent	Power capped below the curve

Cause	Mechanism	Drift type	Pattern in the data
Sensor recalibration	Anemometer replaced or recalibrated; offset or scale change	Virtual and real	Input readings shift, power unchanged
New obstacles / wake effects	New turbines or buildings disturb the flow in some sectors	Real, sudden	Direction-dependent power loss
Control-software update	Changes in pitch or yaw strategy	Real, sudden	Altered curve shape near rated speed

The drift may be sudden, gradual, incremental or recurring (Figure 1.2). It may also be **virtual**, when only the input distribution changes, or **real**, when the conditional relationship between inputs and power changes (Figure 1.3). Real drift is the more damaging because it directly invalidates the learned mapping.

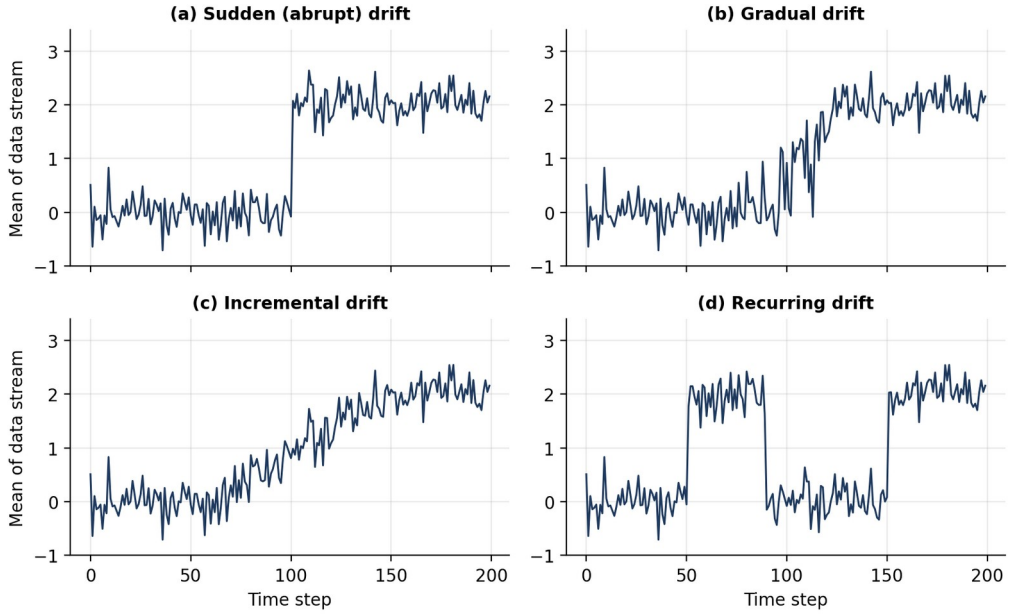


Figure 1.2 Four common temporal patterns of concept drift

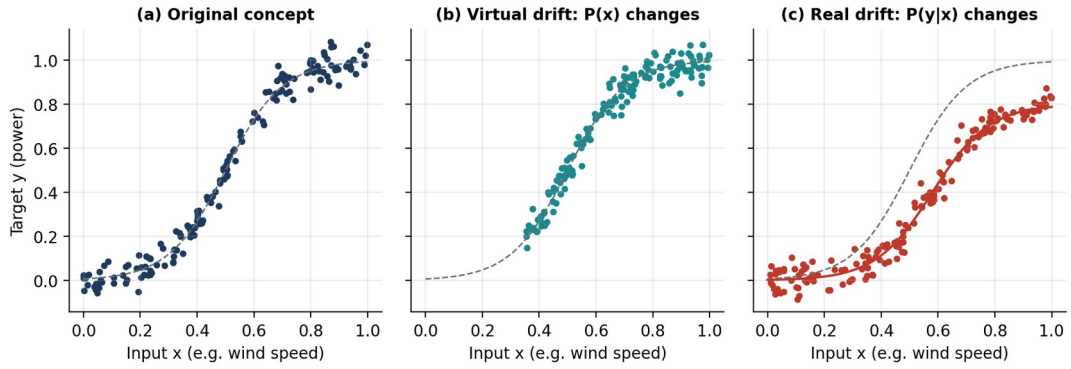


Figure 1.3 Original concept, virtual drift and real drift

1.3 Problem Statement

Wind-power forecasting models trained once on historical data lose reliability when weather regimes and turbine behaviour change. Such models cannot tell when their

learning has become outdated, so forecast errors grow silently. Retraining on a fixed schedule is the common remedy, but it wastes computation when nothing has changed and still reacts late to an abrupt shift. Complex drift-aware deep-learning pipelines exist, yet they add tuning effort, hardware cost and reproducibility burden, and their studies seldom report false alarms, detection delay and retraining cost together with accuracy.

There is therefore a need for a lightweight, interpretable framework that (i) forecasts wind power from power history and weather, (ii) detects when the learned relationships are no longer valid, (iii) adapts the model only when the evidence justifies it, and (iv) quantifies honestly what the adaptation gains and costs.

1.4 Motivation of the Project

- Wind output depends on variables that change over minutes, seasons and years, and small input errors become large power errors because of the cubic law.
- A model trained once assumes stationarity, while ageing, derating, curtailment and sensor changes alter the weather-to-power relationship.
- Recent studies on drift-aware forecasting show that the need is real and active, but most focus on accuracy gains and give less attention to the price of adaptation.
- Adaptation has a cost (false alarms, retraining time), so its benefit must be measured and not assumed.

1.5 Research Question and Hypotheses

The central research question of this project is: **can concept-drift detection and adaptive learning keep wind-power forecasts more reliable under changing conditions than a model trained once, and at what cost?** The question is broken down into four testable hypotheses.

Table 1.3 Research hypotheses

ID	Hypothesis
H1	When the weather-to-power relationship changes, drift-triggered retraining gives a significantly lower forecast error than a static model.
H2	When no drift occurs, the adaptive model raises no alarms and its accuracy equals that of the static model.
H3	Drift-triggered retraining reaches an accuracy comparable to blind weekly retraining while needing far fewer retrainings.
H4	The benefit of adaptation depends on drift magnitude and detector settings; below a certain magnitude it does not pay off.

1.6 Objectives of the Project

1. To build a reproducible data-generation and streaming framework in which concept drift of known type, time and size can be injected into wind-farm data.
2. To engineer features from power history, wind speed, wind direction (as sine and cosine) and temperature, and to train a gradient-boosted forecasting model.
3. To monitor both the input distributions (KS test and PSI) and the standardised prediction-error stream (Page-Hinkley test) in real time.
4. To design an adaptation controller that retrains on a recent, drift-aware window when an alarm fires, with a cool-down guard against repeated retraining.
5. To compare static, adaptive and weekly-retrained models using MAE, RMSE, nRMSE, detection delay, false alarms, retraining frequency and computational cost.
6. To identify when adaptation helps, when it does not, and what it costs, through no-drift controls and sensitivity analysis.

1.7 Scope and Limitations

The project covers one-hour-ahead forecasting of aggregated wind-farm power. Experiments use a controlled synthetic data generator whose physics follow a standard turbine power curve, so that the true drift onset is known and detection delay and false alarms can be measured exactly. The framework itself is data-agnostic: it consumes a time-ordered table of power, wind speed, direction and temperature, and can be applied to measured SCADA or numerical-weather-prediction data without change.

The results presented here therefore establish the behaviour of the method under controlled conditions; they are not a claim about a specific real farm. Validation on measured multi-season turbine or farm data, gradual and recurring drift, probabilistic forecasts and spatial extensions are listed as future work in Chapter 7.

1.8 Contributions

- A modular drift-aware forecasting pipeline: forecaster, error monitor, drift detector, adaptation controller and evaluator.
- A residual-standardisation scheme that removes the dependence of error magnitude on power level so that a single sequential test can monitor the error stream without seasonal false alarms.
- A drift-aware retraining rule that combines a sliding window, recency weighting and down-weighting of data before the estimated change point.
- A controlled evaluation protocol (test-then-train, drift and no-drift scenarios, ten random seeds) that reports accuracy, detection delay, false alarms, retraining frequency and runtime together.

- A sensitivity analysis of detector threshold, cool-down, window length and drift magnitude, with guidance on when adaptation does not pay off.

1.9 Organisation of the Report

Table 1.4 Organisation of the report

Chapter	Title	Content
1	Introduction	Background, problem, objectives, scope and contributions
2	Literature Survey	Forecasting approaches, drift theory and drift-aware studies, research gap
3	Theoretical Background	Wind-power physics, gradient boosting, drift detection and metrics
4	System Analysis and Design	Existing and proposed systems, architecture, DFD, UML, environment
5	Methodology and Implementation	Data generator, features, models, detectors, controller, protocol
6	Results and Discussion	Accuracy, detection, cost, significance, sensitivity, limitations
7	Conclusion and Future Scope	Summary of findings and directions for further work

CHAPTER 2
LITERATURE SURVEY

2.1 Approaches to Wind-Power Forecasting

Wind-power forecasting methods are usually grouped into physical, statistical, machine-learning and deep-learning families. Reviews of the field [9], [10] organise the forecasting pipeline into stages (data preparation, feature construction, model training, post-processing and evaluation) and point out that uncertainty quantification and robustness to changing conditions remain open problems.

2.1.1 Physical and statistical methods

Physical methods use numerical weather prediction (NWP) to estimate wind at hub height and convert it to power through the turbine power curve. They are valuable at longer horizons but are sensitive to NWP bias; for example, an offset in forecast wind speed passes directly into the power forecast [7]. Statistical methods such as persistence, autoregressive integrated moving average (ARIMA) and Kalman filtering exploit the temporal structure of the power series. They are simple and fast but assume linear or locally stationary behaviour.

2.1.2 Machine-learning and deep-learning methods

Support vector regression, random forests and gradient-boosted trees learn the non-linear weather-to-power mapping directly from data and are robust on tabular inputs. Deep networks, including long short-term memory (LSTM) networks, temporal convolutional networks (TCN), attention models and graph networks, add the ability to learn long temporal dependencies and spatial relations between turbines [3], [4], [8]. Their accuracy gains come with more tuning, more data and more computation.

Table 2.1 Comparison of forecasting families

Family	Strengths	Weaknesses	Drift handling
Physical (NWP + curve)	Physically interpretable; good beyond a few hours	Depends on NWP quality; input bias propagates	None unless corrected
Statistical (ARIMA, Kalman)	Fast, few parameters	Linear, locally stationary assumptions	Kalman variants adapt slowly
Classical ML (SVR, RF, GBM)	Non-linear, strong on tabular data	Static after training	Needs external retraining trigger
Deep learning (LSTM, TCN, GNN)	Long dependencies, spatial structure	Data- and compute-hungry; harder to reproduce	Possible with online variants [1], [4]

2.2 Concept Drift: Theory and Detection

The survey of Gama et al. [13] defines concept drift as a change in the joint distribution of inputs and target and gives the standard taxonomy of drift types and adaptation strategies. Later reviews [14] confirm that adaptive learning has four building blocks: memory (which past data to keep), change detection (when to react), learning (how to update) and loss estimation (how to measure the effect). The present project follows exactly this decomposition.

Change-detection methods fall into four families. **Error-rate-based** detectors such as DDM and EDDM watch the prediction error. **Window-based** detectors such as ADWIN [16] compare statistics of two sub-windows of the stream. **Distribution-based** detectors such as the Kolmogorov-Smirnov test [18] and the population stability index (PSI) compare the input distribution of a current window with a reference. **Sequential-analysis** detectors such as CUSUM and the Page-Hinkley test [15] accumulate small deviations from a reference level until a threshold is crossed. Table 2.2 compares them.

Table 2.2 Families of concept-drift detectors

Detector	Monitored quantity	Strengths	Weaknesses
DDM / EDDM	Error rate of a classifier	Simple, online	Designed for classification
ADWIN	Mean of a stream in adaptive windows	Parameter-free windowing, theoretical guarantee	Sensitive to heavy-tailed or heteroscedastic noise
KS test	Input distribution (ECDF gap)	Non-parametric, detects virtual drift	Assumes independent samples; over-fires on autocorrelated data
PSI	Binned input distribution	Interpretable thresholds	Binning choice; no formal test
CUSUM	Standardised deviation from reference	Optimal for a known shift size	Needs tuned drift allowance
Page-Hinkley	Cumulative deviation of the mean	Cheap, gives a change-point estimate	Needs reference level and thresholds

2.3 Drift-Aware Wind and Power Forecasting

He, Yu and Wang [1] propose an online quantile-regression LSTM that detects virtual drift through the Kullback-Leibler divergence of the input distribution and real drift through a change in the continuous ranked probability score (CRPS). Kernel density estimation then yields probabilistic power forecasts. The method handles both input and output drift but is a complex deep pipeline, and the cost of repeated updates is not its focus.

Cao, He, Zhou and Wang [2] study power-load forecasting and show that conventional forecast combinations degrade by about 12% under concept drift and may even do worse than their component models. Their online probabilistic combination framework updates the combination weights continually and gains about 14% in accuracy over the

best single model on real city-level load datasets. Although it addresses load rather than wind data, it demonstrates the failure mode that this project aims to detect.

Cao, He and Zhou [3] extract features online and re-select inputs when drift is detected, feeding a TCN-BiLSTM network for multi-step probabilistic wind-power prediction. Re-selecting features at drift avoids frequent complete model reconstruction, but the hybrid architecture is heavy and harder to reproduce.

Bahi et al. [4] present FreeGNN, a spatio-temporal graph network with a teacher-student scheme, memory replay and drift-aware weighting for continual source-free adaptation. On wind SCADA data it reports an MAE of 0.382 and an RMSE of 0.523, with adaptation taking under 25 ms per batch on a GPU. The authors themselves note the extra compute compared with static models and that subtle drift may be missed.

2.4 Hybrid, Transfer and Domain-Adaptation Approaches

Hou, Yan and Meng [5] switch between a data-driven and a physics-hybrid model when the wind-speed change exceeds a mutation threshold, using adaptive graph convolution on a 99 MW farm. The switching improves stability during abrupt wind changes, but the threshold is fixed and not derived from a statistical drift test. Jin et al. [6] combine dual-stage attention with adaptive transfer learning to forecast power for newly built wind farms that have little history; this is a cold-start problem, not continuous drift monitoring.

Yang et al. [7] make short-term power prediction tolerant to NWP wind-speed offsets and so improve accuracy when the input is biased; the method corrects inputs but does not reveal when the learned mapping itself has changed. Yang, Liu and Yu [8] introduce SA-STGCN, a spectral-attentive spatio-temporal graph convolutional network with wavelet multi-scale learning, aimed at accuracy rather than drift adaptation.

Two reviews [9], [10] consolidate probabilistic-forecast metrics and the key processes of wind forecasting respectively, without proposing a drift-handling method. Finally, source-free time-series domain adaptation methods such as LEMON [11] (wavelet-based multi-scale temporal imputation) and TFDA [12] (time-frequency synergy with contrastive learning) adapt a trained model to a new domain without access to source data. They address domain shift in general time series rather than continual drift in wind power, and they give no retraining trigger or cost analysis.

2.5 Summary of Surveyed Works

Table 2.3 Summary of surveyed works (1 of 2)

Ref.	Year	Method	Key finding	Limitation / gap
[1]	2025	O-DDA-QRLSTM; KL divergence and CRPS for virtual / real drift	Handles input and output drift; probabilistic forecasts	Complex deep pipeline; update cost not studied
[2]	2025	Online probabilistic forecast combination (load)	Ensembles degrade ≈12% under drift; framework gains ≈14%	Load, not wind; ensemble adds compute
[3]	2026	Online feature extraction + TCN-BiLSTM	Re-selecting features at drift avoids full rebuild	Heavy hybrid; hard to reproduce
[4]	2026	FreeGNN with replay and drift-aware weighting	MAE 0.382, RMSE 0.523; < 25 ms per batch	Extra compute; subtle drift may be missed
[5]	2025	Adaptive graph convolution; physics / data-driven switching	More stable under abrupt wind change	Fixed threshold, no statistical test
[6]	2025	Dual-stage attention + adaptive transfer	Forecasts for new farms with scarce data	Cold start, not continuous monitoring

Table 2.4 Summary of surveyed works (2 of 2)

Ref.	Year	Method	Key finding	Limitation / gap
[7]	2025	Power prediction tolerant to NWP speed error	Better accuracy under speed offset	Corrects inputs; no drift trigger
[8]	2025	SA-STGCN with wavelet multi-scale learning	Models spatial and multi-scale temporal structure	Accuracy-oriented; no adaptation focus
[9]	2025	Review of uncertainty modelling and metrics	Consolidates probabilistic methods and metrics	Review; no drift method
[10]	2025	Review of key forecasting processes	Organises stages and open problems	Review; not drift-adaptive
[11]	2025	LEMON: source-free domain adaptation	Adapts without source data	General series; domain shift, not drift
[12]	2025	TFDA: time-frequency contrastive adaptation	Improves source-free adaptation	Not wind-specific; no retraining cost

2.6 Research Gap and Positioning of This Work

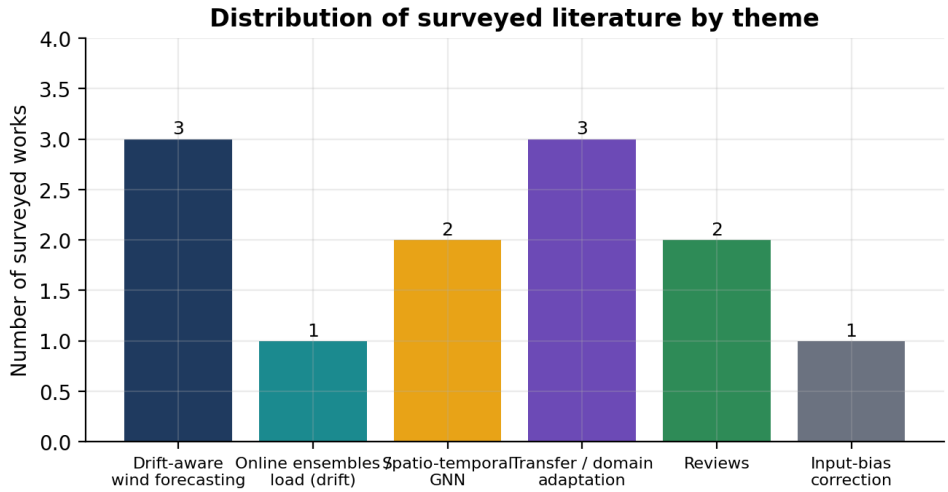


Figure 2.1 Distribution of the twelve surveyed works by theme

The survey shows that drift-aware forecasting is an active need (Figure 2.1) but that existing solutions are either architecture-heavy, tied to a fixed threshold, or aimed at a different problem such as cold start or domain shift. Three gaps stand out.

1. **No lightweight, interpretable trigger.** Deep drift-aware pipelines [1], [3], [4] raise accuracy but add tuning effort and compute. A standard regressor with a statistical alarm is rarely studied.
2. **Adaptation is assumed to help.** Studies report accuracy gains but seldom report false alarms, detection delay and retraining cost, or show scenarios in which adaptation brings nothing.
3. **Evaluation hides temporal change.** Random train/test splits make reported accuracy look better than it will be after a regime shift.

Table 2.5 Capability comparison with surveyed approaches

Capability	[1]	[3]	[4]	[5]	[7]	This work
Detects input drift	Yes	Yes	Partly	Fixed rule	No	Yes
Detects error / real drift	Yes	Yes	Yes	No	No	Yes
Lightweight standard regressor	No	No	No	No	Yes	Yes
Reports detection delay	No	No	No	No	No	Yes
Reports retraining cost	No	No	Partly	No	No	Yes
No-drift control scenario	No	No	No	No	No	Yes

Entries reflect what the surveyed abstracts state and are indicative; "No" means not reported in the abstract.

This project addresses these gaps by treating adaptation as a hypothesis to be tested: it measures accuracy, detection speed and computational cost together, and includes no-drift controls that expose false alarms and wasted retraining.

CHAPTER 3
THEORETICAL BACKGROUND

3.1 Wind-Power Conversion

The kinetic power of air of density ρ flowing at speed v through a rotor of swept area A is ½ρAv³. A turbine captures a fraction C_p of it (the power coefficient, bounded by the Betz limit of 16/27 ≈ 0.593) [17]:

$$P = \tfrac{1}{2} \cdot \rho \cdot A \cdot C_p \cdot v^3 \tag{3.1}$$

Air density follows the ideal-gas law from pressure p and absolute temperature T, with the specific gas constant of dry air $R_s = 287.05\ \mathrm{J\ kg^{-1}\ K^{-1}}$:

$$\rho = p / (R_s \cdot T) \tag{3.2}$$

Colder air is denser, so the same wind carries more power in winter; this is why temperature is an input of the model. The operating behaviour of a real turbine is summarised by its power curve, which is modelled here by the piecewise function:

$$P(v) = 0 \text{ for } v < v_{in} \text{ or } v \geq v_{out};\ P_r \cdot (v^3 - v_{in}^3) / (v_r^3 - v_{in}^3) \text{ for } v_{in} \leq v < v_r;\ P_r \text{ for } v_r \leq v < v_{out} \tag{3.3}$$

where $v_{in} = 3$ m/s is the cut-in speed, $v_r = 12$ m/s the rated speed, $v_{out} = 25$ m/s the cut-out speed and $P_r = 2000$ kW the rated power. A first-order sensitivity analysis of Equation (3.1) gives:

$$\Delta P / P \approx 3 \cdot \Delta v / v \tag{3.4}$$

so a 10% error in wind speed produces a power error of roughly 33%, as noted in Chapter 1.

3.2 Forecasting Problem Formulation

Let y_t be the farm power in hour t and x_t the vector of features available when the forecast for hour t is issued (weather inputs of hour t and lagged power and speed). The task is one-step-ahead regression:

$$\hat{y}_t = f_\theta(x_t), \quad x_t = [v_t, \sin d_t, \cos d_t, T_t, y_{t-1}, y_{t-2}, y_{t-24}, \bar{y}_{t-1:t-6}, v_{t-1}, \bar{v}_{t-1:t-3}] \tag{3.5}$$

The function f_θ is trained on a window of past pairs (x, y). If the joint distribution P(x, y) is the same during training and use, the trained model generalises. Concept drift is the failure of this condition.

3.3 Gradient-Boosted Regression Trees

Gradient boosting [19] builds an additive model of shallow regression trees h_m. At stage m the model adds a tree fitted to the negative gradient (the residuals, for squared loss) of the current model, shrunk by a learning rate ν:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x), \quad h_m \approx \operatorname{argmin} \Sigma_i \, w_i \cdot (r_i - h(x_i))^2 \tag{3.6}$$

where $r_i = y_i - F_{m-1}(x_i)$ are the residuals and w_i are optional sample weights. Histogram-based gradient boosting, as implemented in scikit-learn [21] and inspired by LightGBM [20], first bins every feature into at most 255 discrete values, which makes split search very fast and keeps the cost of a retraining small; this is essential for an approach that retrains repeatedly. Sample weights let the learner emphasise recent observations, a property exploited by the adaptation controller (Section 5.7).

3.4 Formal Definition of Concept Drift

Following Gama et al. [13], concept drift between time t and t + Δ means that the joint distribution changes:

$$P_t(x, y) \neq P_{t+\Delta}(x, y) \tag{3.7}$$

Because P(x, y) = P(x)·P(y | x), two cases are distinguished. In **virtual drift** only P(x) changes (for example a windier season) and a model that has learned P(y | x) over the whole input range remains valid. In **real drift** P(y | x) changes (for example turbine derating) and the learned mapping becomes wrong even when the inputs look familiar. Table 3.1 relates the types to wind-power examples used in this project.

Table 3.1 Drift types and wind-power examples

Type	Definition	Wind-power example	Injected here
Sudden / abrupt	Change at one instant	New obstacle creating a wake sector	Yes (week 28)
Gradual	Old and new concepts alternate	Intermittent curtailment	No (future work)
Incremental	Slow continuous change	Blade erosion, anemometer ageing	No (future work)
Recurring	Previous concept returns	Seasonal regimes	Natural seasonality only
Virtual	P(x) changes, P(y|x) fixed	Windier month	Natural seasonality
Real	P(y|x) changes	Derating, wake loss	Yes

3.5 Drift-Detection Methods

3.5.1 Standardised error stream

The absolute forecast error grows with the power level because both measurement noise and model error are larger on the steep part of the power curve. A seasonal change in the share of high-wind hours would therefore change the mean error even without drift. To remove this effect, the absolute error is divided by the mean absolute error expected at the forecast level $\hat{y}_t$, estimated from validation data:

$$z_t = |y_t - \hat{y}_t| / m(\hat{y}_t) \tag{3.8}$$

where $m(\cdot)$ is a step function over six forecast-level bins (Section 5.5). By construction z_t has mean $\mu_0 \approx 1$ and standard deviation σ_0 when the model is valid.

3.5.2 Page-Hinkley test

The Page-Hinkley test [15] detects an increase in the mean of a stream. With reference level μ_0, tolerance δ and threshold λ, it computes:

$$m_t = \Sigma_{i \le t} (z_i - \mu_0 - \delta), \quad M_t = min_{i \le t}\, m_i, \quad alarm\ when\ m_t - M_t > \lambda \tag{3.9}$$

While the stream stays at its reference mean, the term $-\delta$ pulls m_t downwards and M_t follows it, so the statistic stays near zero. After a mean shift of size $\Delta > \delta$ the statistic grows at rate $\Delta - \delta$ per step and crosses λ after about $\lambda / (\Delta - \delta)$ steps. The time at which m_t reached its minimum is a natural estimate of the change point:

$$\hat{\tau} = argmin_{i \le t}\, m_i \tag{3.10}$$

This estimate is used by the controller to decide which training samples predate the drift.

3.5.3 CUSUM and ADWIN

The one-sided CUSUM statistic accumulates standardised deviations and resets at zero: $g_t = \max(0, g_{t-1} + (z_t - \mu_0) / \sigma_0 - k)$, with alarm when $g_t > h$. ADWIN [16] keeps a window W of recent values and declares drift when two sub-windows W_0 and W_1 have means that differ by more than a Hoeffding-type bound:

$$|\hat{\mu_0} - \hat{\mu_1}| > \varepsilon_{cut} = \sqrt{((1 / 2m) \cdot ln(4 / \delta'))}, \quad m = 1 / (1/n_0 + 1/n_1), \quad \delta' = \delta / n \tag{3.11}$$

Both are used as alternative error-stream detectors in the comparison of Section 6.7.

3.5.4 Kolmogorov-Smirnov test and PSI

For two samples with empirical distribution functions F_1 and F_2, the KS statistic and the large-sample rejection rule are:

$$D = sup_x |F_1(x) - F_2(x)|, \quad reject\ if\ D > \sqrt{(-½\ ln(\alpha/2))} \cdot \sqrt{((n_1 + n_2) / (n_1 \cdot n_2))} \tag{3.12}$$

The PSI compares binned proportions r_i (reference) and c_i (current) over B bins:

$$PSI = \Sigma_{i=1}^{B} (c_i - r_i) \cdot \ln(c_i / r_i) \quad (3.13)$$

Conventionally PSI < 0.1 means a negligible shift, 0.1 to 0.25 a moderate shift and > 0.25 a major shift. Figure 3.1 illustrates both statistics. Hourly wind speed is strongly autocorrelated, so the effective sample size is much smaller than the number of hours and the nominal KS p-value is over-confident; Section 6.4 shows the practical consequence.

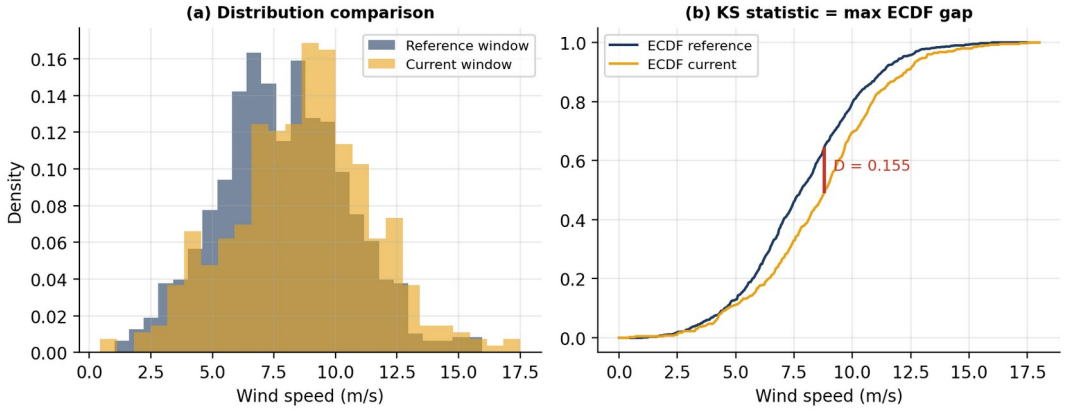


Figure 3.1 Distribution comparison and the KS statistic as the largest gap between two ECDFs

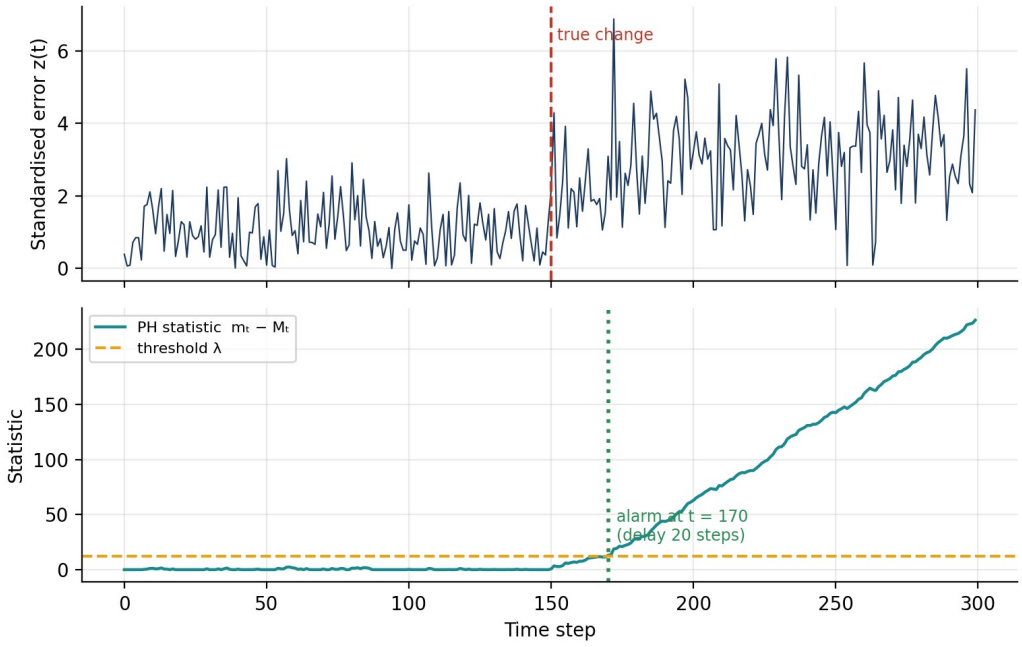


Figure 3.2 Page-Hinkley test on a synthetic stream with a mean shift at step 150

3.6 Drift-Aware Retraining

When an alarm fires at time k, the model is refitted on the last W hours with sample weights that combine recency decay and down-weighting of data older than the estimated change point:

$$w_i = 2^{-(k-i)/h} \cdot \kappa_i, \quad \kappa_i = \kappa_{pre} \text{ if } i < \hat{\tau} - 12, \text{ otherwise } 1 \quad (3.14)$$

where h is the half-life of the recency decay, 336 hours, and $\kappa_{pre} = 0.08$. Section 5.7 explains the choice of these values.

3.7 Evaluation Metrics

Forecast accuracy is measured with the mean absolute error, the root mean squared error and the RMSE normalised by rated capacity [22]:

$$MAE = (1/N)\,\Sigma\,|y_t - \hat{y}_t|, \quad RMSE = \sqrt{((1/N)\,\Sigma\,(y_t - \hat{y}_t)^2)}, \quad nRMSE = RMSE / P_r \times 100\% \tag{3.15}$$

Drift handling is measured with four further quantities. The **detection delay** is the time from the true drift onset to the first alarm. A **false alarm** is an alarm raised before the onset (or, in the no-drift scenario, any alarm). The **retraining frequency** is the number of model updates, and the **computational cost** is the cumulative wall-clock time of retraining and monitoring.

3.8 Prequential (Test-Then-Train) Evaluation

In stream learning the standard protocol is prequential evaluation [23]: every observation is first used to test the current model, and only afterwards used for training. The forecast for hour t is made before y_t is known; once y_t arrives it contributes to the error statistics and to the monitors. This avoids look-ahead leakage and mirrors operation in a control room.

3.9 Statistical Tests for Comparison

Because the three models are evaluated on the same ten random streams, differences in MAE are compared with a paired two-sided t-test and, as a non-parametric check, the Wilcoxon signed-rank test. A difference is called significant at the 5% level ($p < 0.05$).

CHAPTER 4
SYSTEM ANALYSIS AND DESIGN

4.1 Existing System Overview

In current practice, wind-power forecasting systems are built in two ways. In the first, a model (statistical, machine-learning or deep-learning) is trained once on a historical period and then deployed unchanged. In the second, the model is retrained at regular calendar intervals (daily, weekly or monthly) regardless of its recent behaviour. Both are simple to operate, and both are widely used.

The static model is cheap but silent: when the wind regime or the turbine behaviour changes, nothing tells the operator that the forecast has become unreliable. The scheduled model keeps itself fresh but pays the retraining cost every period even if nothing has changed, and it still reacts late to an abrupt change that happens just after a retraining. Neither design provides evidence about whether a change has actually occurred.

Table 4.1 Disadvantages of the existing systems

Existing practice	Disadvantage
Train once, deploy unchanged	Error grows silently after drift; no alarm, no evidence of when the model became outdated
Blind periodic retraining	Wastes computation when stationary; late reaction to abrupt change; short training windows can raise error when nothing has changed
Complex deep drift-aware pipelines	Higher tuning effort, hardware and reproducibility cost; update cost rarely reported
Accuracy-only reporting	Hides false alarms, detection delay and retraining cost, so the real price of adaptation is unknown
Random train/test splits	Optimistic accuracy because temporal change between train and test is ignored

4.2 Proposed System Overview

The proposed system keeps a standard gradient-boosted regressor but surrounds it with a monitoring and adaptation layer. Every hour the system forecasts the next power value, compares it with the actual value when it arrives, converts the error into a standardised score, and feeds that score to a Page-Hinkley detector. In parallel, a daily check compares the recent wind-speed distribution against a reference with the KS test and PSI. When the error evidence shows that the learned relationship no longer holds, an adaptation controller retrains the model on a recent, drift-aware window and swaps it in. A cool-down guard prevents repeated retraining within 48 hours.

Two control models run on exactly the same data: a static model that is never updated and a model that is retrained blindly every week. Comparing the adaptive model with both separates the benefit of adapting at all (against static) from the benefit of adapting only when needed (against weekly).

4.3 Advantages of the Proposed System

- **Evidence-based adaptation:** the model is updated only when a statistical test indicates that its error has risen.
- **Interpretable and lightweight:** a standard regressor, a one-line sequential test and a small controller replace a complex deep pipeline.
- **Honest evaluation:** drift and no-drift scenarios, ten seeds, and reporting of delay, false alarms and runtime.
- **Controlled cost:** a cool-down guard and the standardised error stream limit the number of retrainings.
- **Data-agnostic:** the same pipeline can be applied to measured SCADA or NWP data.

4.4 System Architecture

Figure 4.1 shows the architecture. Data flow from the acquisition module (M1) through preprocessing (M2) to the forecast model (M3). The prediction and error-monitor module (M4) emits the residual stream. The drift detector (M5) consumes the standardised errors and the input windows and raises alarms to the adaptation controller (M6), which retrains the model in M3. The evaluation module (M7) compares the adaptive model with the static and weekly-retrained control models.

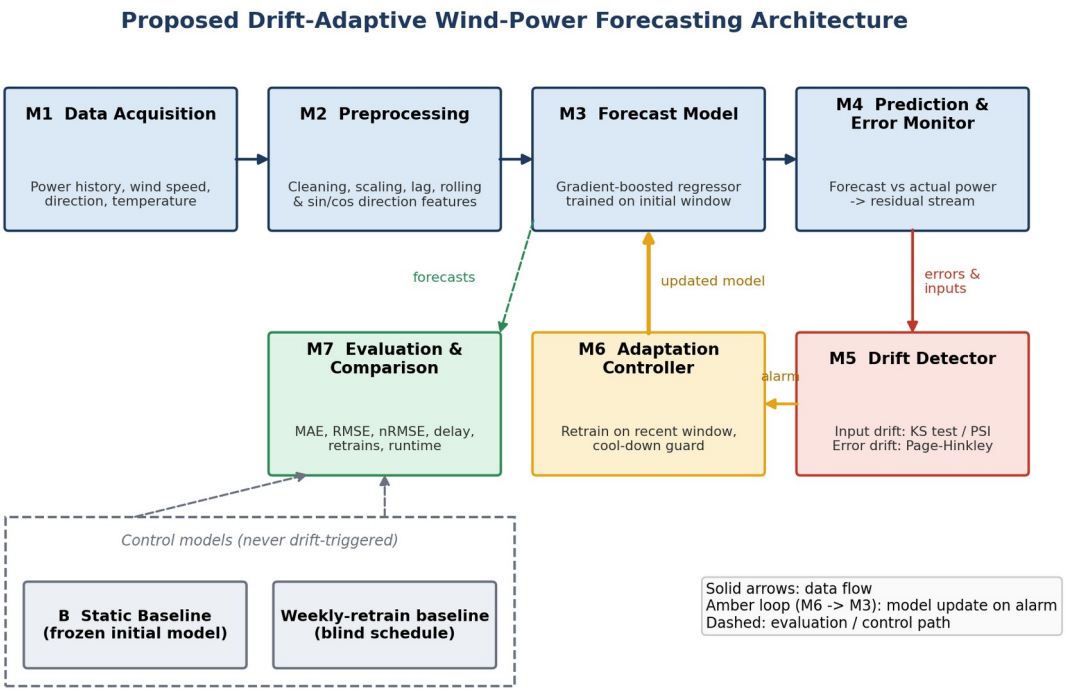


Figure 4.1 Architecture of the drift-adaptive wind-power forecasting system

Table 4.2 Functional modules of the proposed system

Module	Name	Responsibility	Output
M1	Data acquisition	Read or generate hourly power, speed, direction, temperature	Time-ordered table
M2	Preprocessing	Sine/cosine direction, lags, rolling means	Feature matrix X
M3	Forecast model	Histogram gradient-boosted regressor	Forecast $\hat{y}(t)$
M4	Prediction & error monitor	Compute residual and standardised score	z(t) stream
M5	Drift detector	Page-Hinkley on z(t); KS/PSI on inputs	Alarm and change-point estimate
M6	Adaptation controller	Cool-down check, window and weights, retrain	Updated model
M7	Evaluation	MAE, RMSE, nRMSE, delay, retrains, runtime	Result tables and figures

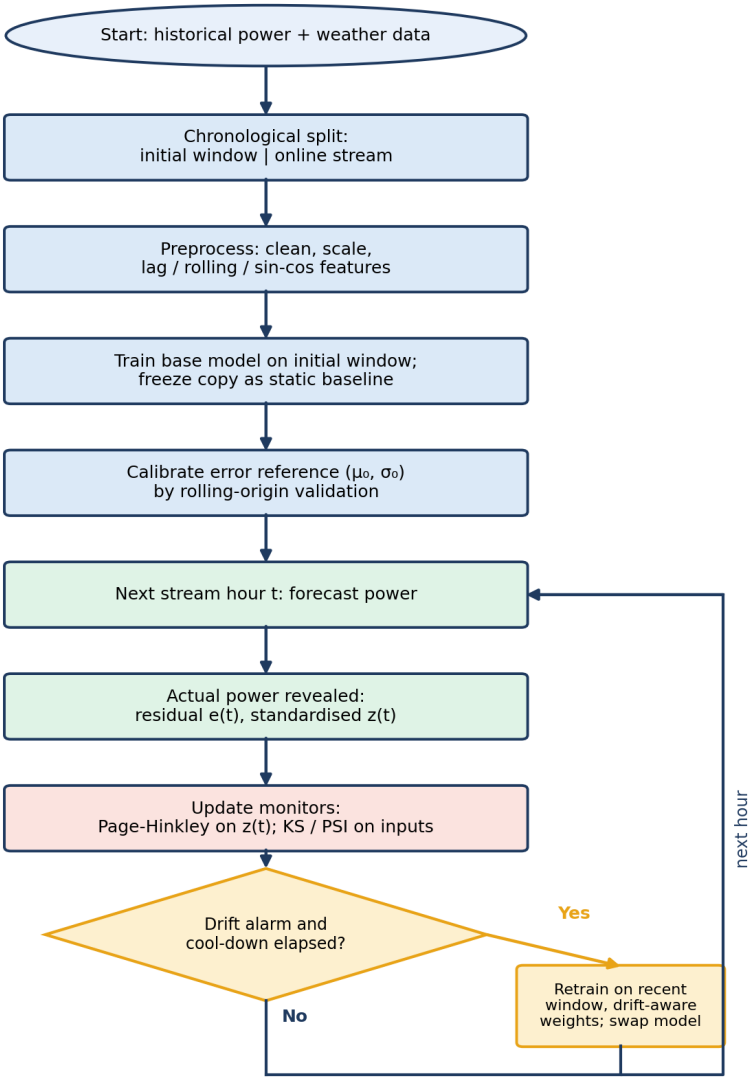


Figure 4.2 Flowchart of the online forecasting and adaptation loop

4.5 Data Flow Diagram

The level-1 data flow diagram in Figure 4.3 shows six processes and two data stores. Raw data are preprocessed into the feature store (D1). The trained model produces forecasts that are sent to the operator and to the monitor. The detection process raises an alarm that restarts the training process, and every alarm and metric is written to the log (D2), from which the evaluation process builds reports.

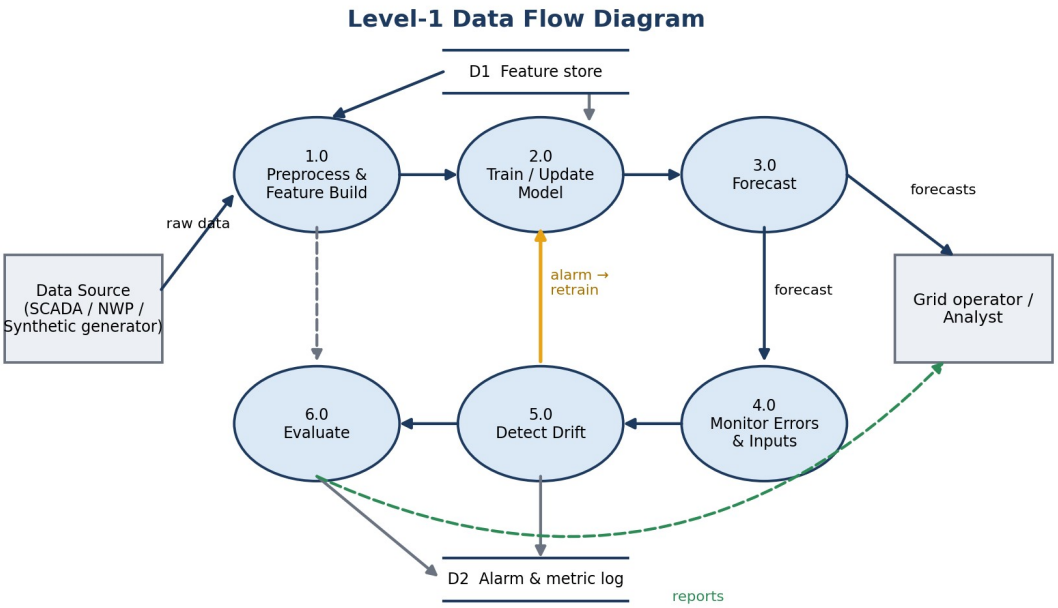


Figure 4.3 Level-1 data flow diagram

4.6 UML Design

The use-case diagram (Figure 4.4) shows two actors: the data scientist, who prepares data, trains models and compares results, and the grid operator, who consumes forecasts and reports. The sequence diagram (Figure 4.5) shows one pass of the loop, including the alarm and retraining messages. The class diagram (Figure 4.6) lists the main software classes.

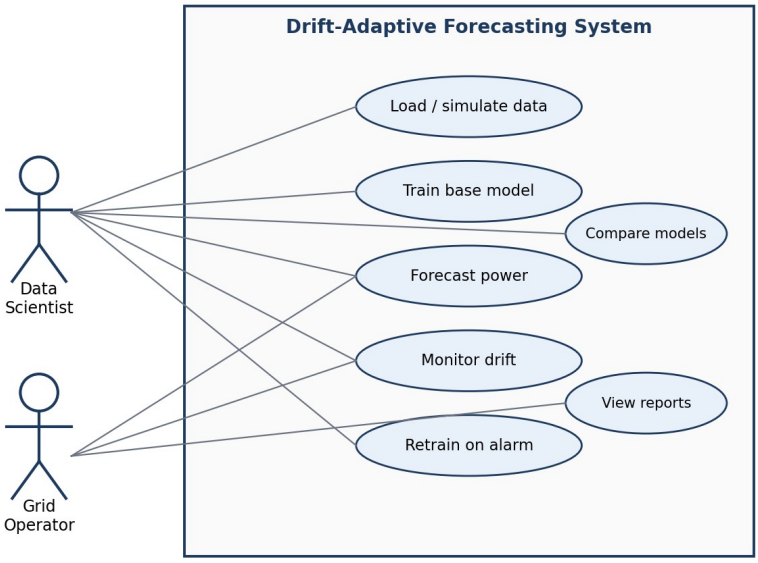


Figure 4.4 Use-case diagram

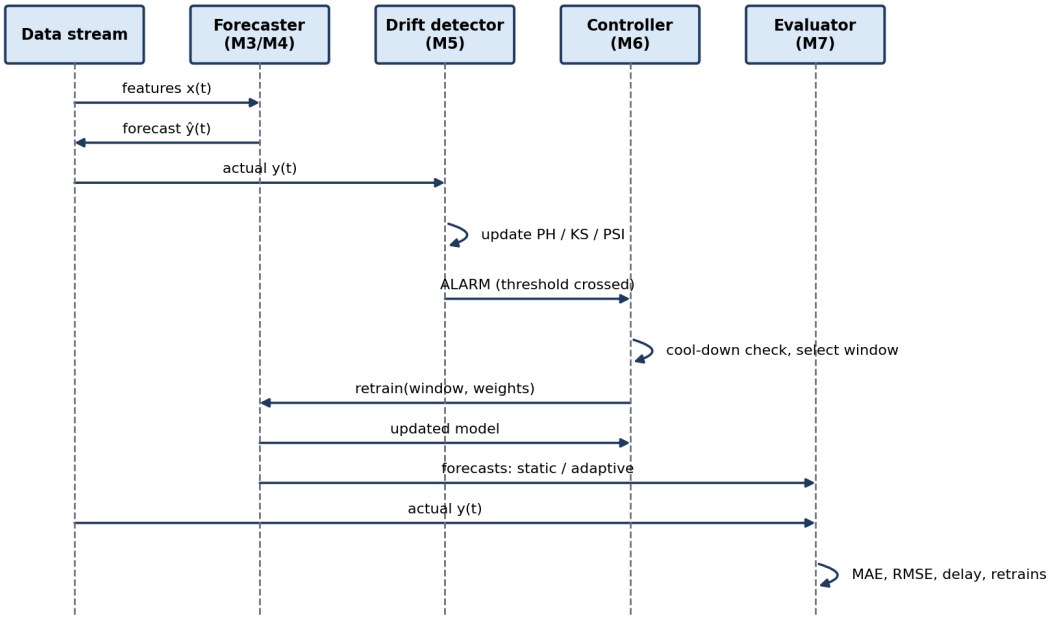


Figure 4.5 Sequence diagram of one forecasting and adaptation cycle

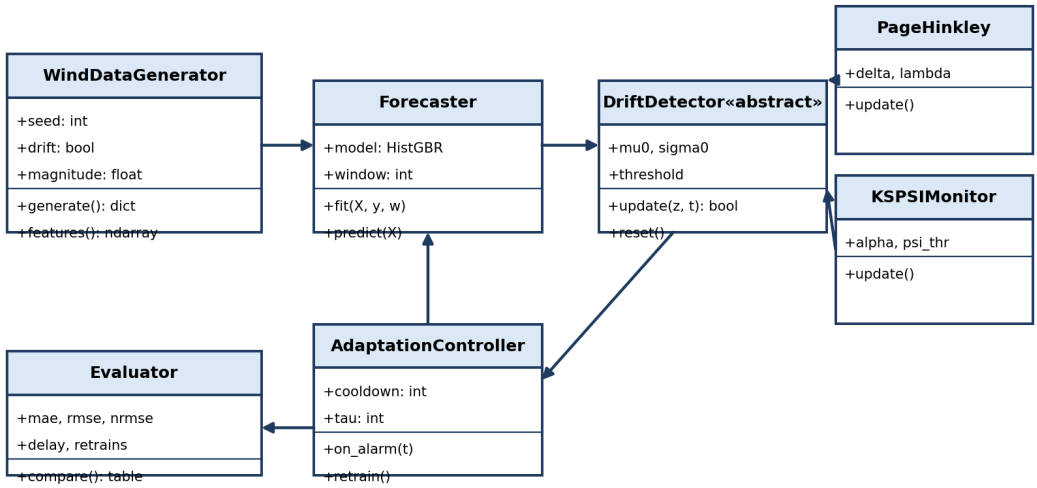


Figure 4.6 Class diagram of the main software components

4.7 Implementation Environment

The system is implemented in Python. All experiments run on a CPU; no GPU is needed because histogram gradient boosting on a few hundred to a few thousand rows trains in a fraction of a second. Table 4.3 lists the software used.

Table 4.3 Software environment

Component	Version	Purpose
Python	3.12	Implementation language
NumPy [25]	2.4	Array computation, data generator
pandas	3.0	Tabular handling and summaries

Component	Version	Purpose
SciPy [26]	1.17	KS test, paired t-test, Wilcoxon test
scikit-learn [21]	1.8	HistGradientBoostingRegressor, permutation importance
River [24]	0.26	ADWIN detector for comparison
Matplotlib [27]	3.10	All figures
Node.js docx	—	Generation of this report

The complete ten-seed experiment (20 stream runs, each with three models) takes about five minutes on a single CPU core, and the sensitivity studies add roughly twenty minutes, which shows that the approach is practical for modest hardware.

4.8 Project Schedule

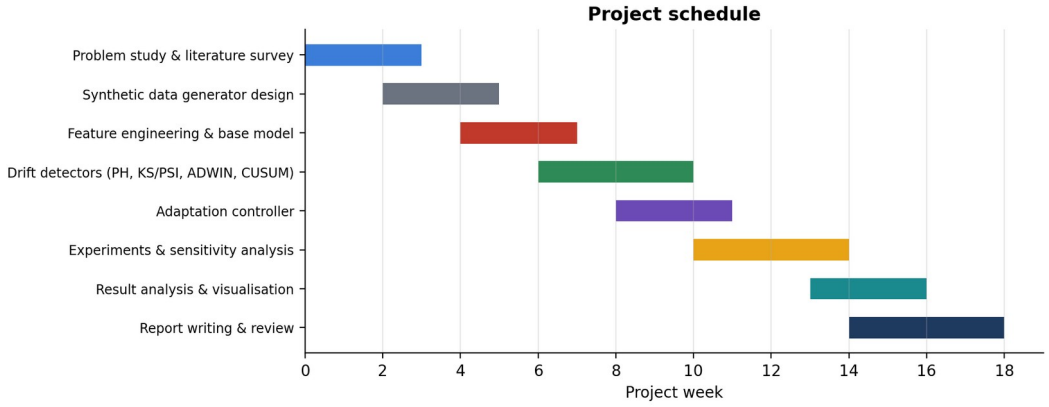


Figure 4.7 Project schedule over eighteen weeks

CHAPTER 5
METHODOLOGY AND IMPLEMENTATION

5.1 Data Description and Synthetic Generator

Real farm-level data with a known drift time are rare, and without a known drift time neither detection delay nor false alarms can be measured. The experiments therefore use a controlled synthetic generator whose physical structure follows the power-conversion model of Chapter 3. For every random seed, the generator produces 106 weeks (17,808 hourly samples) of wind speed, wind direction, temperature and power for a 2 MW aggregated unit.

5.1.1 Weather model

Wind speed is the sum of a seasonal term (windier in winter), a diurnal term and an autocorrelated Gaussian process; temperature has an annual and a daily cycle plus autocorrelated noise; wind direction fluctuates around a south-westerly prevailing direction:

$$v_t = max(0,\ 7.6 + 1.3\ cos(2\pi(doy - 20)/365) + 0.7\ sin(2\pi(h - 14)/24) + 2.7 \cdot u_t) \quad (5.1)$$

$$d_t = (225° + 55° \cdot u'_t)\ mod\ 360° \quad (5.2)$$

Here u_t and u'_t are unit-variance AR(1) processes with autocorrelation 0.96 and 0.97 respectively, doy is the day of year and h the hour of day. The generated wind speed has a realistic persistence: successive hours are strongly correlated, which is also the reason for the autocorrelation caveat of the KS test (Chapter 3). Figure 5.1 shows three weeks of the resulting stream and Figure 5.2 shows the weekly profile over the whole horizon.

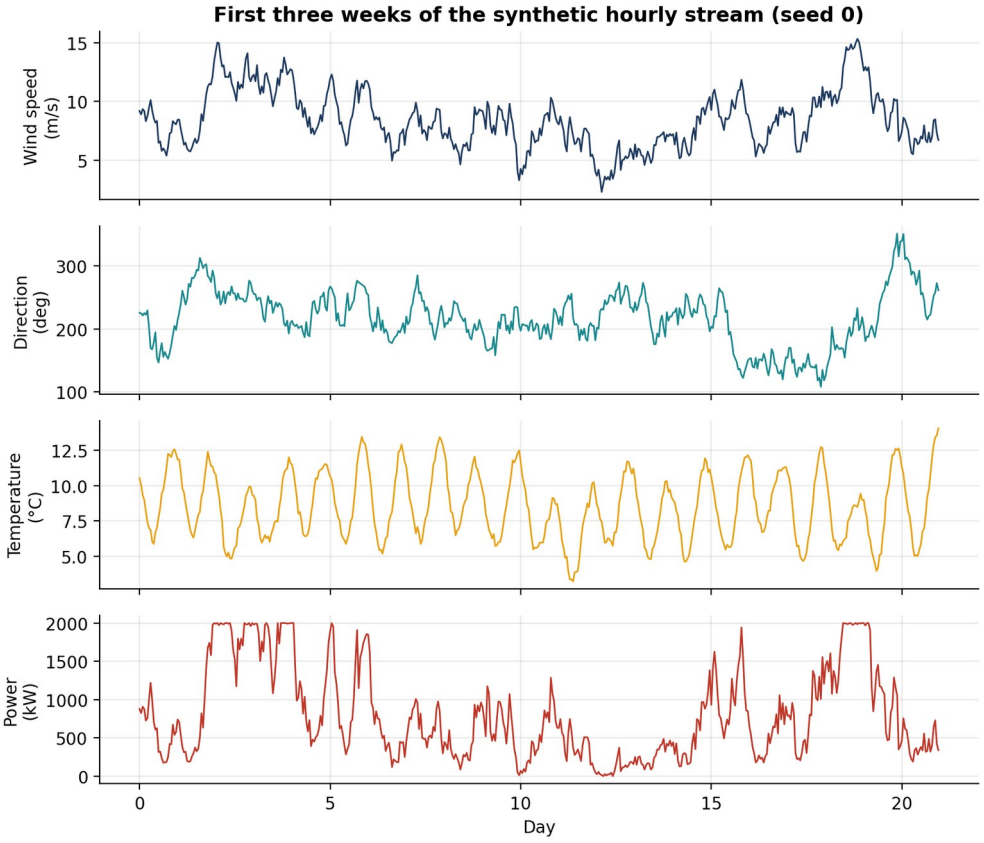


Figure 5.1 First three weeks of the synthetic hourly stream (seed 0)

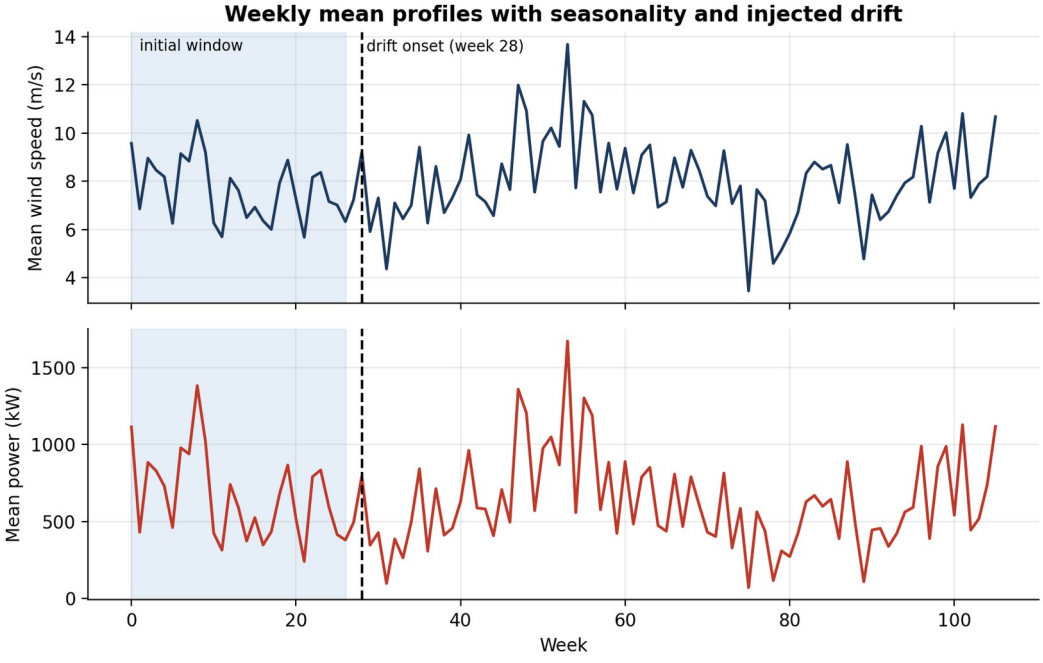


Figure 5.2 Weekly mean wind speed and power over the 106-week horizon with injected drift

5.1.2 Power model and noise

Power is computed from the true wind speed through the piecewise power curve of Equation (3.3), corrected by the air-density ratio ρ/1.225, and then perturbed by

multiplicative noise (σ = 0.8%) and additive noise (σ = 7 kW). The measured wind speed given to the model is the true speed plus sensor noise (σ = 0.15 m/s).

5.1.3 Injected drift

At week 28 (hour 4,704), a sudden drift is injected that has one real and one virtual component. The real component reduces turbine output by 7.5% at all wind speeds (ageing or derating) and by a further 8.5% when the wind comes from the sector 200° to 265° (a new obstacle creating a wake). The virtual component multiplies the measured wind speed by 1.04 (an anemometer recalibration), so that the model sees higher speeds for the same physical wind. Figure 5.3 shows the effect on the power curve: after the drift the same measured wind speed produces less power, and the maximum output falls from 2000 kW to about 1850 kW. Figure 5.4 shows the wake-affected sector.

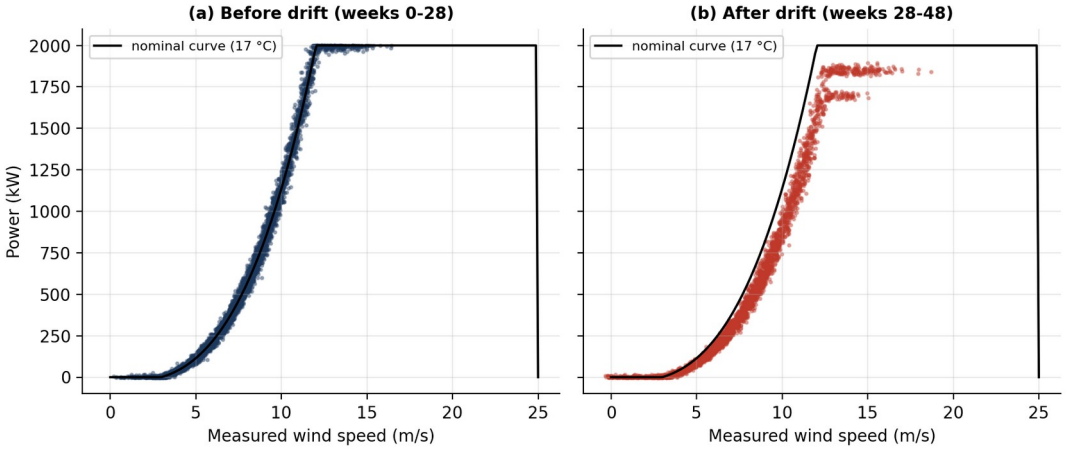


Figure 5.3 Measured wind speed versus power before and after the injected drift

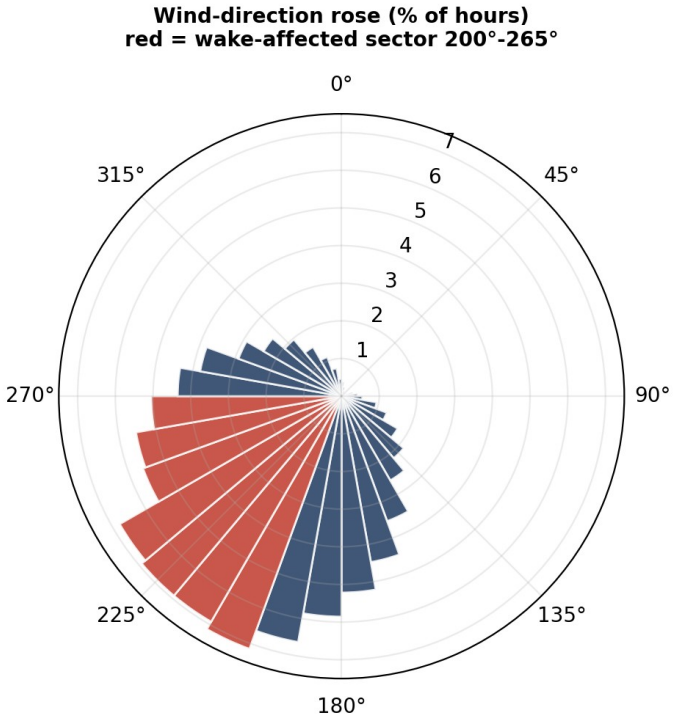


Figure 5.4 Wind-direction rose; the wake-affected sector is shown in red

Table 5.1 Parameters of the synthetic data generator

Parameter	Value	Parameter	Value
Rated power P_r	2000 kW	Mean wind speed	7.6 m/s
Cut-in / rated / cut-out speed	3 / 12 / 25 m/s	Seasonal / diurnal amplitude	1.3 / 0.7 m/s
Total horizon	106 weeks (17,808 h)	Speed AR(1) φ, σ	0.96, 2.7 m/s
Initial window	26 weeks (4,368 h)	Direction mean, σ	225°, 55°
Online stream	80 weeks (13,440 h)	Temperature (annual / daily)	17 ± 9 °C / ± 3 °C
Drift onset	Week 28 (hour 4,704)	Power noise (mult. / add.)	0.8% / 7 kW
Output loss, all speeds	7.5%	Sensor noise (speed / direction)	0.15 m/s / 4°
Extra loss, sector 200°-265°	8.5%	Anemometer scale	× 1.04
Random seeds	0 to 9	Scenarios	Drift and no drift

The same generator with the drift switched off provides the no-drift control scenario. Because the seeds fix every random number, each scenario is exactly reproducible.

Figure 5.5 confirms an important property of the data: even without any injected drift, the wind-speed distribution of a later eight-week period differs significantly from that of the initial window, because of natural seasonality and persistence (KS p-values of the order 10^{-19} to 10^{-31}). Input-distribution tests alone therefore cannot separate harmless variability from damaging drift, a point discussed in Chapter 6.

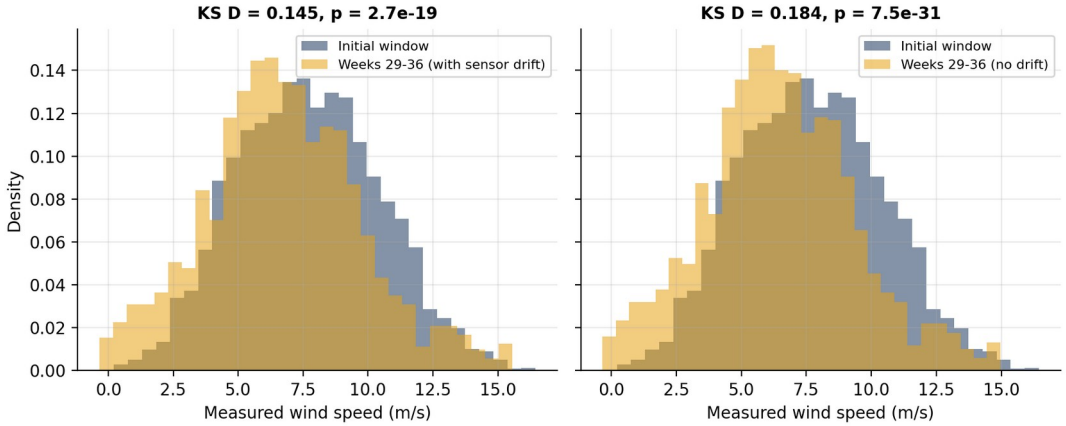


Figure 5.5 Wind-speed distribution of the initial window against weeks 29-36, with and without injected drift

5.2 Preprocessing and Feature Engineering

Ten features are built for every hour (Table 5.2). Wind direction is an angle, so its raw value is discontinuous at 0°/360°: 359° and 1° are close physically but far apart numerically. It is therefore encoded as the pair (sin, cos) (Figure 5.6). Lagged and rolling power features exploit the strong persistence of wind output. All lags use only past values, so no future information leaks into the features.

Table 5.2 Feature set used by the forecaster

No.	Feature	Definition	Role
1	v	Measured wind speed at hour t	Primary driver (cubic law)
2-3	sin d, cos d	Cyclic encoding of wind direction	Wake-sector and regime information
4	T	Air temperature	Air density correction
5-6	P(t−1), P(t−2)	Power one and two hours ago	Short-term persistence
7	P(t−24)	Power 24 hours ago	Diurnal pattern
8	mean P(t−6..t−1)	Rolling mean of the last six hours	Smoothed recent level
9	v(t−1)	Wind speed one hour ago	Wind tendency
10	mean v(t−3..t−1)	Rolling mean of the last three hours	Smoothed wind level

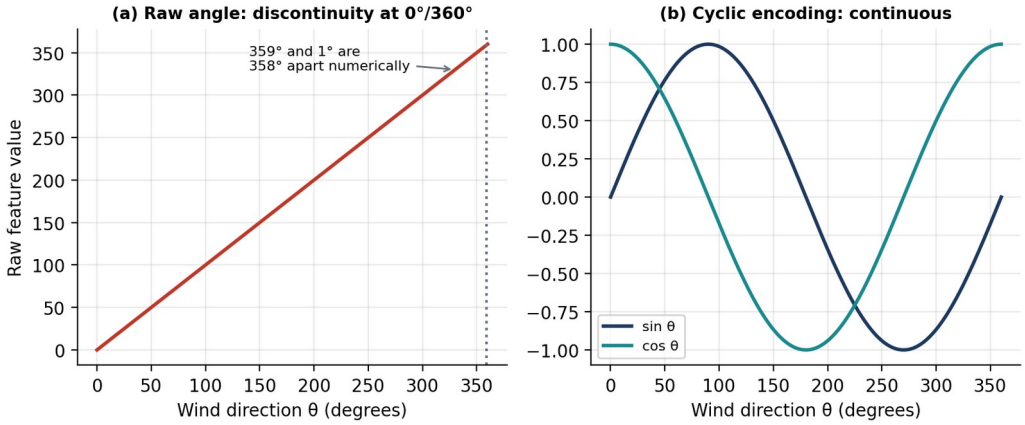


Figure 5.6 Raw angle versus sine/cosine encoding of wind direction

Figure 5.7 shows the Pearson correlations in the initial window. Power correlates almost perfectly with its own recent lags (0.95 with P(t−1)) and with wind speed (0.95), while temperature and direction are weakly correlated individually; their value lies in interaction effects that tree models can capture.

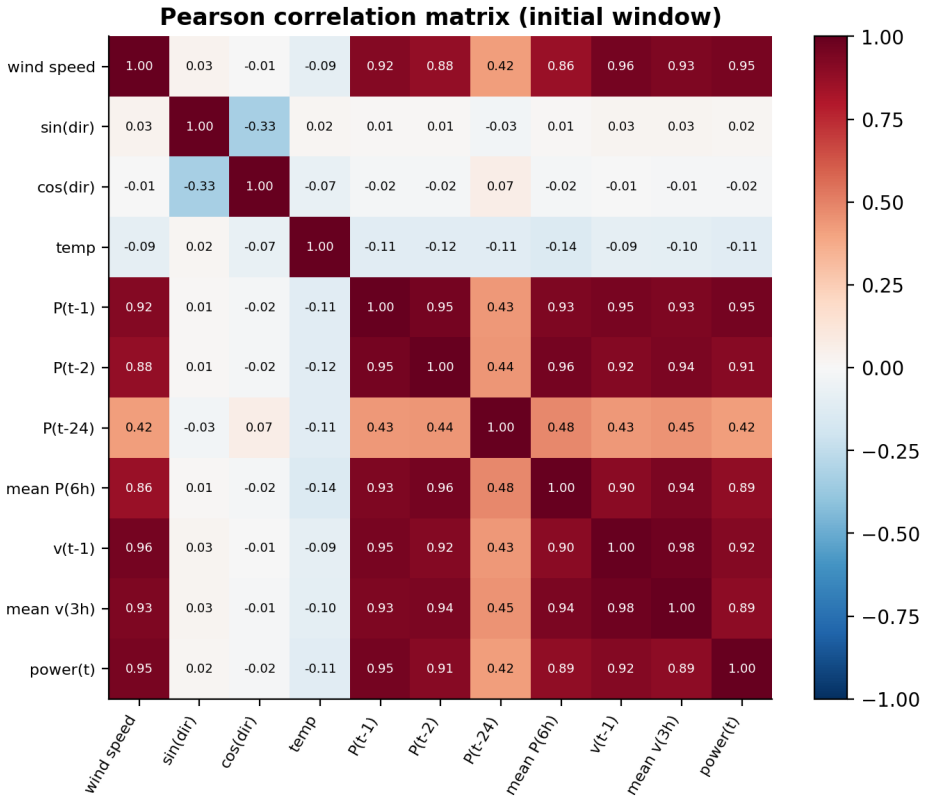


Figure 5.7 Correlation matrix of features and target in the initial window

5.3 Chronological Split and Streaming Protocol

The data are never shuffled. The first 26 weeks form the **initial window** used to train the base model and to calibrate the error monitor. The remaining 80 weeks form the **online stream**, replayed hour by hour in test-then-train order (Figure 5.8). The drift onset lies 2

weeks into the stream, so the detector must work from an essentially fresh state, which is a demanding case.

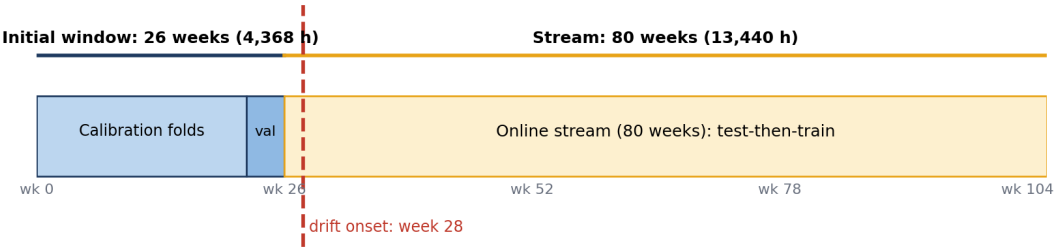


Figure 5.8 Chronological split, online stream and drift onset

5.4 Base Forecasting Model

The forecaster is scikit-learn's HistGradientBoostingRegressor [21]. The same configuration is used by all three models so that differences come only from the update policy. The initial model is trained once on the whole initial window with equal sample weights.

Table 5.3 Hyper-parameters of the forecaster

Parameter	Value	Comment
max_iter (trees)	120	Boosting rounds
learning_rate ν	0.1	Shrinkage
max_depth	6	Controls interaction order
min_samples_leaf	20	Regularisation for small retraining windows
loss	Squared error	Targets conditional mean
random_state	seed	Reproducibility

Permutation importance on eight held-out weeks (Figure 5.9) ranks wind speed first (MAE increases by 429.2 kW when it is shuffled), followed by v(t-1) (2.2 kW) and P(t-1) (0.7 kW). Features with a small importance in the pre-drift period, such as direction, matter later because the injected wake loss is direction-dependent: they carry the information the model needs once it is retrained.

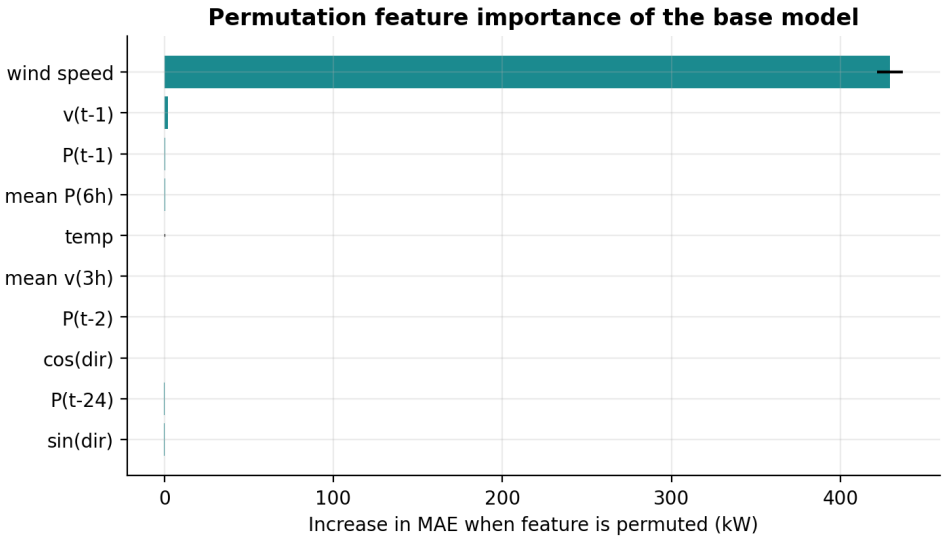


Figure 5.9 Permutation importance of the base model features

5.5 Error Standardisation and Calibration

A single sequential test on raw absolute error would be fooled by seasonal changes in power level, because the error is larger when the wind is strong. The system therefore calibrates a heteroscedasticity map inside the initial window by **rolling-origin validation**: at origins in weeks 10, 14, 18 and 22, a model is trained on the preceding four weeks (using the same weighted procedure as a retraining) and forecasts the following four weeks. The pooled forecasts are cut into six quantile bins of the forecast level, and the mean absolute error of each bin is stored (Figure 5.10). The monitored score is then z_t from Equation (3.8).

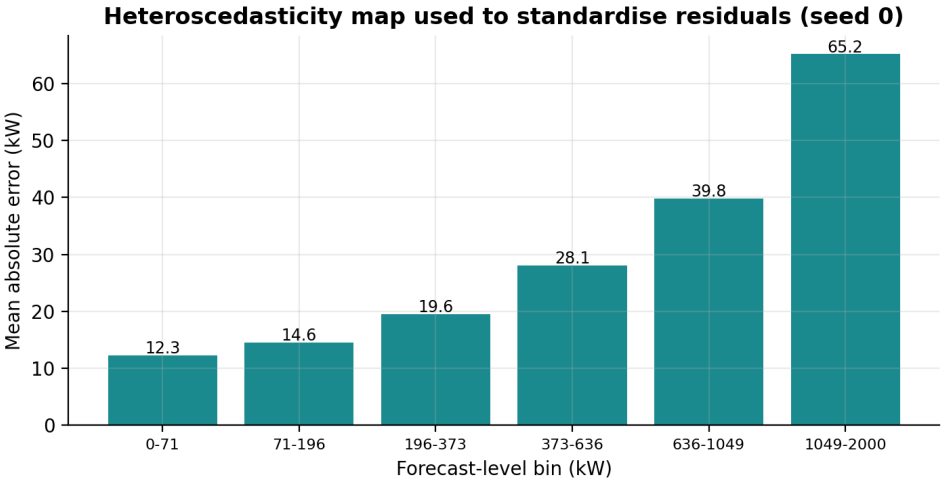


Figure 5.10 Mean absolute error per forecast-level bin (seed 0), used to standardise residuals

For seed 0 the six bin errors are 12.3, 14.6, 19.6, 28.1, 39.8, 65.2 kW. The pooled score has mean $\mu_0 = 1.00$ and standard deviation $\sigma_0 = 0.80$. Using a model trained like a retrained model for the calibration, rather than the long-window initial model, is

deliberate: it sets the reference at the error level that a freshly adapted model can achieve, so a successful adaptation brings the score back below the alarm region.

5.6 Drift Detectors

5.6.1 Error-stream detector (Page-Hinkley)

The Page-Hinkley test of Equation (3.9) runs on z_t with reference μ_0, tolerance $\delta = 0.6\,\sigma_0$ and threshold $\lambda = 15\,\sigma_0$. The tolerance keeps the statistic near zero when the model is valid; the threshold sets how much accumulated excess error is needed before an alarm. These values were chosen so that no false alarm occurred in the no-drift calibration runs while the delay stayed within one to two days.

5.6.2 Input-distribution monitors (KS and PSI)

Once per day, the wind speed of the last 168 hours is compared with the preceding four weeks (lagged by one week) using the KS test and the PSI. An input alarm is considered only if $p < 10^{-6}$ and PSI > 0.25. Because natural variability regularly satisfies these conditions (Section 6.4), an input alarm is **accepted only when it is corroborated by degraded accuracy**, i.e. the 24-hour mean of z_t exceeds $\mu_0 + 2\sigma_0$. The input monitors thus act as a second, advisory channel and cannot trigger retraining on their own.

5.6.3 Decision rule

```
for each stream hour t:
    ŷ_t   = model.predict(x_t)               # forecast first
    z_t   = |y_t - ŷ_t| / m(ŷ_t)             # standardised error
    alarm = PageHinkley.update(z_t)
    if t is the end of a day:
        if KS(ref, cur).p < 1e-6 and PSI(ref, cur) > 0.25 and mean(z[-24:]) > mu0 +
2*sigma0:
            alarm = alarm or "input"
    if alarm and (t - last_retrain) >= COOLDOWN:   # 48 h
        retrain(window=672 h, weights=w_i)         # Section 5.7
        last_retrain = t ; PageHinkley.reset()
    elif alarm:
        PageHinkley.reset()                        # suppressed by cool-down
```

5.7 Adaptation Controller

When an alarm passes the cool-down check, the controller retrains the forecaster on the most recent $W = 672$ hours (four weeks) ending at the alarm hour. Three design decisions govern the training set.

1. **Sliding window.** Only the last four weeks are used, so that old regimes are forgotten.
2. **Recency weights.** Each sample receives weight $2^{-\text{age}/336}$, a half-life of two weeks.

3. **Drift-aware weights.** Samples older than the estimated change point $\hat{\tau} - 12$ hours (Equation (3.10)) are multiplied by 0.08, so the few post-change samples dominate even when an alarm fires soon after the change.

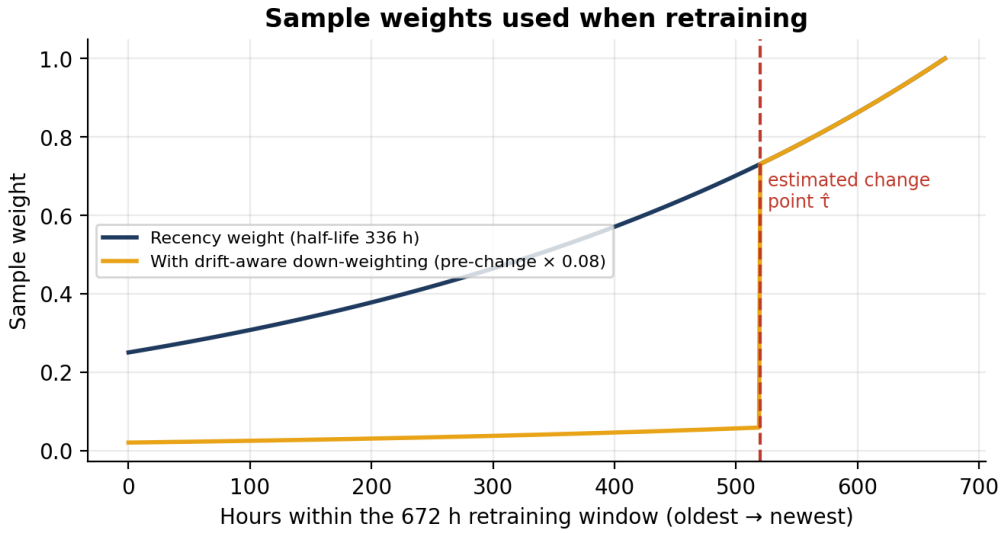


Figure 5.11 Sample weights used when retraining, with and without drift-aware down-weighting

The **cool-down** of 48 hours prevents a second retraining before the first has had time to take effect. After a retraining, the Page-Hinkley statistic is reset, so the monitor re-arms on the new model.

5.8 Baseline Models

Table 5.4 Models compared in every scenario

Model	Update policy	Purpose of comparison
Static	Trained once on the initial window; never changed	Shows the damage caused by drift and the value of any adaptation
Weekly retrain	Retrained at the end of every week on the last 672 h (same weights, no drift awareness)	Shows whether adapting only when needed matches blind adaptation, and at what cost
Adaptive (proposed)	Retrained on alarm from the Page-Hinkley / input monitors with cool-down	Proposed approach

5.9 Module-Wise Implementation

5.9.1 M1 Data acquisition

The function generate(seed, drift, magnitude) returns hourly arrays of measured speed, true speed, direction, temperature, observed power and noise-free power, together with the drift onset hour. For measured data the same dictionary can be filled from a SCADA or NWP file.

5.9.2 M2 Preprocessing

The function features(D) builds the ten features of Table 5.2. It uses only past values for lags and rolling means, and handles the first hours by back-filling.

5.9.3 M3 Forecast model

new_model(seed) and fit_recent(...) construct and fit the regressor. fit_recent applies the recency and drift-aware sample weights.

5.9.4 M4 Prediction and error monitor

Inside run_adaptive the forecasts are produced in blocks of 24 hours for speed; if an alarm fires within a block, the remaining forecasts of the block are discarded and recomputed with the new model, which keeps the result identical to a strictly hour-by-hour replay.

5.9.5 M5 Drift detector

PageHinkley, Cusum and AdwinDet share one interface (update, reset). The input monitors use scipy.stats.ks_2samp and the psi function.

5.9.6 M6 Adaptation controller

Cool-down bookkeeping, window selection, weight construction and model swap are implemented in run_adaptive; the retraining time is measured with a high-resolution timer.

5.9.7 M7 Evaluation

The function metrics returns MAE, RMSE and nRMSE; experiment collects detection delay, false alarms, retraining counts and timings; the aggregation script computes means, standard deviations and statistical tests over seeds.

5.10 Experimental Protocol

Table 5.5 Experimental protocol

Item	Setting
Scenarios	Drift (abrupt at week 28) and no drift
Seeds	Ten independent streams (0 to 9) per scenario
Forecast horizon	One hour ahead
Evaluation	Prequential, on the 80-week stream (13,440 forecasts per run)
Models	Static, adaptive, weekly retrain
Primary metrics	MAE, RMSE, nRMSE (% of 2000 kW)
Drift metrics	Detection delay, false alarms, retrainings, retraining time
Sensitivity studies (5 seeds)	Detector type, threshold λ, cool-down, retraining window, drift magnitude

Item	Setting
Statistics	Paired t-test and Wilcoxon signed-rank test over seeds

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Overview of the Experiments

All results in this chapter come from running the code listed in Appendix B. Two scenarios were run for each of ten random seeds: a **drift scenario** with the abrupt drift of Section 5.1 injected at week 28, and a **no-drift scenario** that is identical except for the drift. In every run the static, adaptive and weekly-retrained models forecast the same 13,440 hourly values of the 80-week stream. Values are reported as mean ± standard deviation over the ten seeds unless stated otherwise.

6.2 Forecast Accuracy under Concept Drift

Table 6.1 gives the accuracy of the three models in the drift scenario. The static model reaches an MAE of 134.7 ± 5.6 kW. The adaptive model reduces it to 33.7 ± 2.1 kW, a reduction of 75.0%, and the RMSE falls from 178.4 ± 5.6 kW to 53.0 ± 3.7 kW. Expressed as a percentage of the 2000 kW rated power, the nRMSE improves from 8.92% to 2.65%. The reduction was between 72.6% and 76.4% in every one of the ten seeds, so the result is not an artefact of a single lucky stream.

Table 6.1 Forecast accuracy in the drift scenario (mean ± std over 10 seeds)

Model	MAE (kW)	RMSE (kW)	nRMSE (%)	Retrainings	Retraining time (s)
Static	134.7 ± 5.6	178.4 ± 5.6	8.92 ± 0.28	0	0.0
Adaptive (proposed)	**33.7 ± 2.1**	**53.0 ± 3.7**	**2.65 ± 0.19**	**16.2 ± 6.6**	**1.98 ± 0.82**
Weekly retrain	35.3 ± 1.3	62.7 ± 5.4	3.14 ± 0.27	80	9.59 ± 0.53

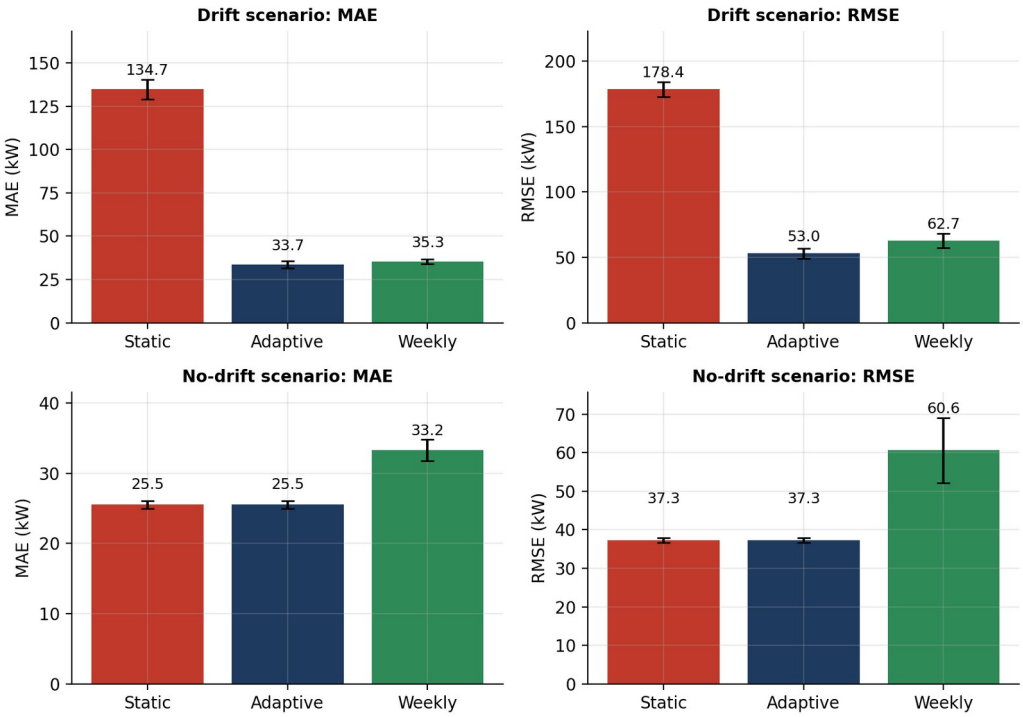


Figure 6.1 MAE and RMSE of the three models in both scenarios (error bars: one standard deviation)

The adaptive model is also slightly more accurate than blind weekly retraining (33.7 kW against 35.3 kW, 4.4% lower), with a larger advantage in RMSE (53.0 kW against 62.7 kW). The reason is explained below: the weekly schedule reacts to the drift only at the next weekly boundary and has no information about when the change occurred, whereas the adaptive model reacts within about a day and then weights the post-change data heavily.

6.2.1 Behaviour over time

Figure 6.2 plots the weekly MAE of seed 0. Before the drift all three models behave alike. At week 28 the error of the static model jumps and stays high, with a mean level of about 130 to 140 kW and large week-to-week swings that depend on how much strong wind occurs. The adaptive and weekly models show a short transient and then return to the pre-drift error level. The amber lines mark the retraining events of the adaptive model; they are concentrated in the first days after the onset.

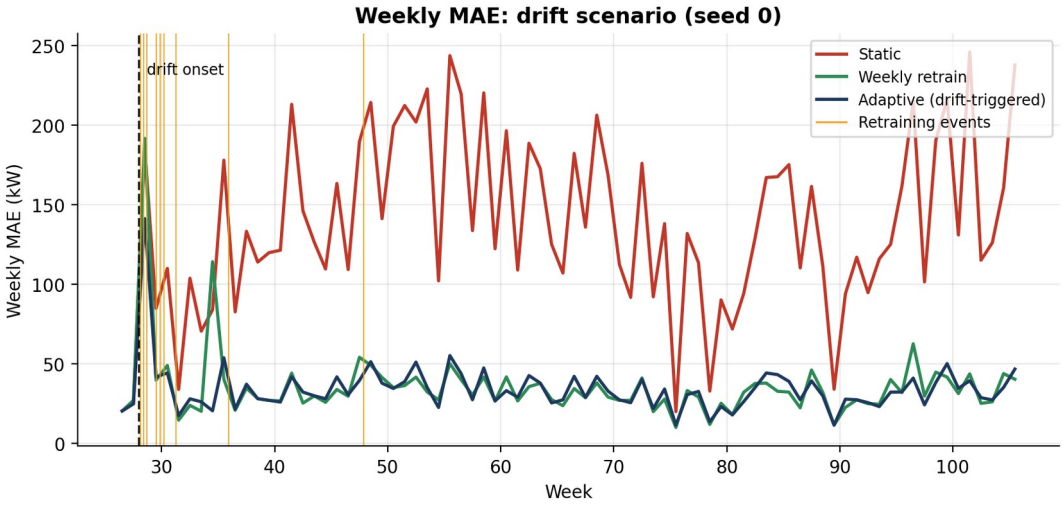


Figure 6.2 Weekly MAE of the three models in the drift scenario (seed 0)

The cumulative absolute error (Figure 6.3) shows the practical consequence. The static model keeps accumulating error at a steady, high rate, while the adaptive and weekly models accumulate error at a rate close to their pre-drift level. By the end of the stream the static model has accumulated about four times as much absolute error.

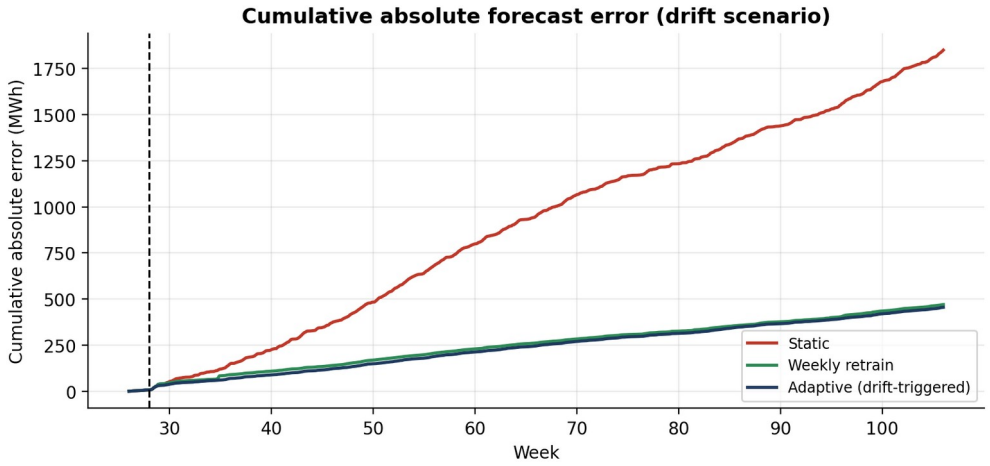


Figure 6.3 Cumulative absolute forecast error in the drift scenario (seed 0)

6.2.2 The transient after the onset

The first two weeks after the onset are the hard part. For seed 0, the mean absolute error in the 14 days after the onset is 135.4 kW for the static model, 115.6 kW for weekly retraining and 91.3 kW for the adaptive model. The adaptive model is therefore the quickest to recover, although it has not yet reached its steady-state accuracy. After the transient, the figures are 140.7 kW (static), 33.2 kW (weekly) and 32.7 kW (adaptive). Figure 6.4 zooms into the days around the onset: the static model over-predicts during every high-wind period, because it still assumes the old power curve.

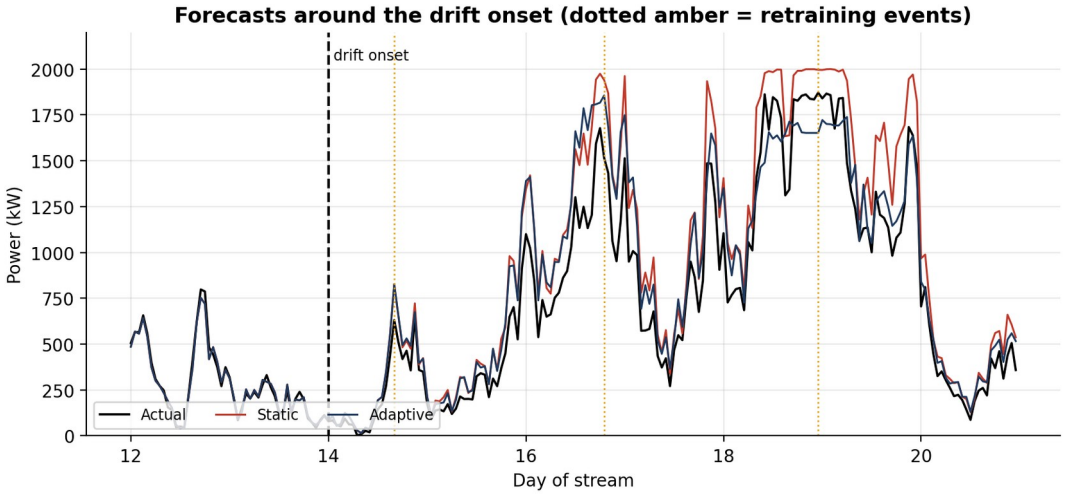


Figure 6.4 Actual and forecast power around the drift onset (seed 0)

6.2.3 Quality of the forecasts

Figure 6.5 compares predicted with actual power after the transient. The static forecasts lie systematically above the diagonal, especially at medium and high power (R^2 = 0.911), whereas the adaptive forecasts scatter closely around the diagonal (R^2 = 0.992). The residual distributions in Figure 6.6 show the same: the static residuals are shifted to the positive side with a standard deviation of about 119 kW, while the adaptive and weekly residuals are centred on zero with standard deviations of 49 kW and 55 kW.

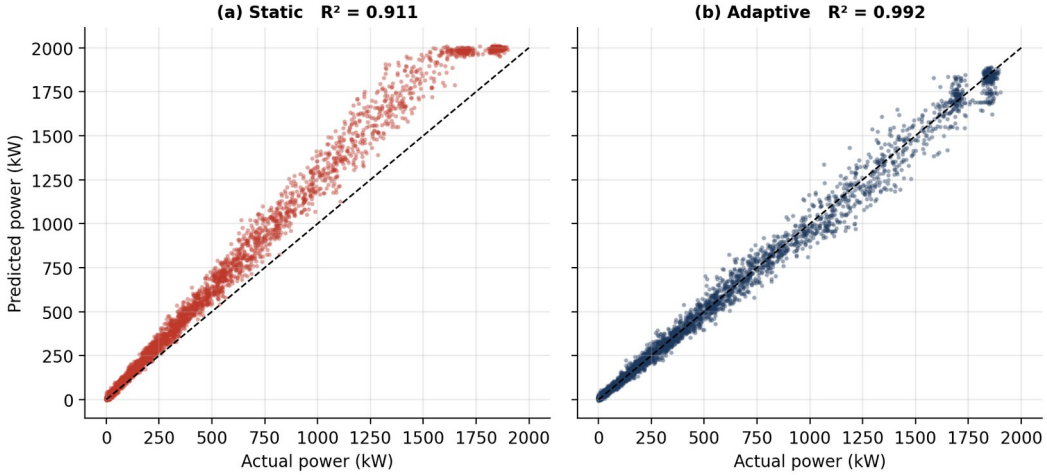


Figure 6.5 Predicted versus actual power after the drift transient (seed 0)

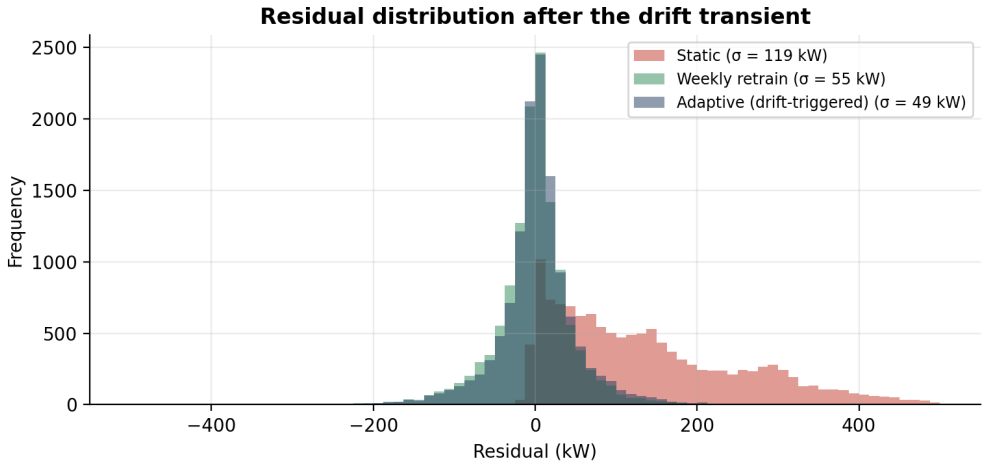


Figure 6.6 Residual distributions after the drift transient (seed 0)

Figure 6.7 and Table 6.2 locate the loss. In the steep region of the power curve (8 to 12 m/s) the static model's MAE rises from 32 and 59 kW before the drift to 173 and 313 kW afterwards, because the cubic law amplifies the 4% speed-scale error and the power loss. The adaptive model brings these bins back to 39 and 65 kW. Above rated speed the static model is wrong by about 200 kW because the real maximum output has fallen to about 1850 kW.

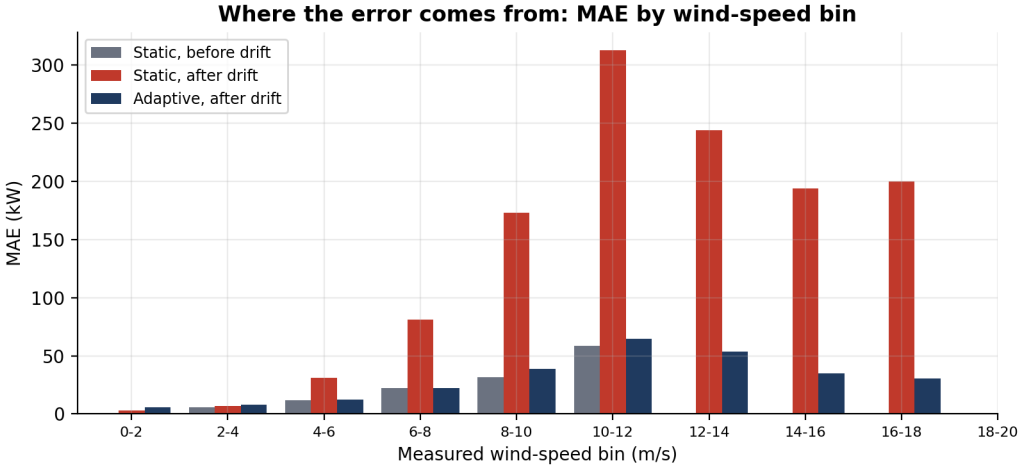


Figure 6.7 MAE by wind-speed bin before and after the drift (seed 0)

Table 6.2 MAE by wind-speed bin (kW, seed 0)

Speed bin (m/s)	Static, before drift	Static, after drift	Adaptive, after drift
0-2	–	3.3	5.7
2-4	6.0	7.1	7.9
4-6	11.7	30.8	12.6
6-8	22.4	81.3	22.0
8-10	31.8	173.0	38.6
10-12	58.8	312.5	64.6
12-14	–	243.6	53.6

Speed bin (m/s)	Static, before drift	Static, after drift	Adaptive, after drift
14-16	–	194.0	34.9
16-18	–	200.0	30.4

Dashes: fewer than ten samples in the bin (before drift, the highest speeds occurred rarely).

6.3 Forecast Accuracy without Drift

The no-drift scenario tests whether adaptation does harm when nothing changes. Table 6.3 shows that the adaptive model raised no alarm in any of the ten seeds, so it never retrained and its forecasts are identical to those of the static model (MAE 25.5 ± 0.6 kW for both). Weekly retraining, in contrast, degrades the MAE to 33.2 ± 1.5 kW, 30.2% worse, and the RMSE from 37.3 ± 0.7 to 60.6 ± 8.4 kW. The cause is the short four-week retraining window: a model trained on four weeks has fewer examples of rare high-wind states than a model trained on 26 weeks, and its variance is higher. Retraining on a schedule is therefore not only wasteful when nothing has changed; it can reduce accuracy.

Table 6.3 Forecast accuracy in the no-drift scenario (mean ± std over 10 seeds)

Model	MAE (kW)	RMSE (kW)	nRMSE (%)	Retrainings	Alarms
Static	25.5 ± 0.6	37.3 ± 0.7	1.87 ± 0.03	0	–
Adaptive (proposed)	**25.5 ± 0.6**	**37.3 ± 0.7**	**1.87 ± 0.03**	**0**	**0 in 10 seeds**
Weekly retrain	33.2 ± 1.5	60.6 ± 8.4	3.03 ± 0.42	80	–

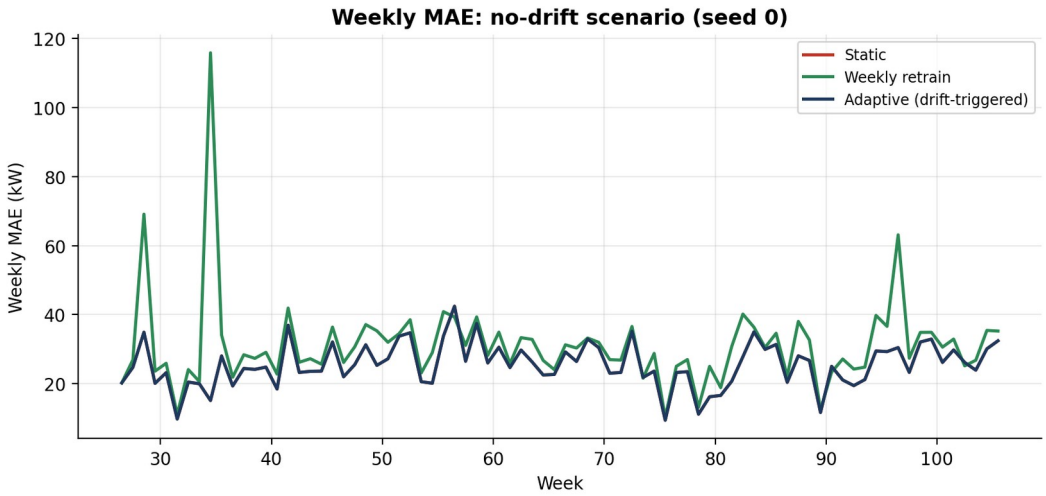


Figure 6.8 Weekly MAE of the three models in the no-drift scenario (seed 0)

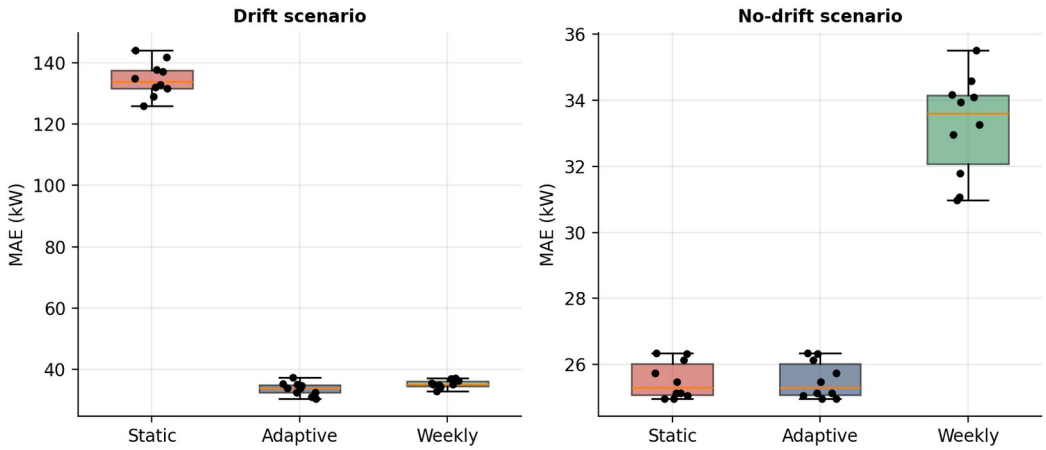


Figure 6.9 Distribution of the MAE over the ten seeds in both scenarios

6.4 Drift-Detection Performance

6.4.1 Detection delay

The drift was detected in all 10 of 10 seeds. The mean delay from the true onset to the first alarm was 21.5 hours (median 15.5 h, standard deviation 20.7 h, range 2 to 68 h) (Table 6.4, Figure 6.10). Detection is not instantaneous because the drift has no effect on the error while the wind is weak: when the wind is below the cut-in speed or the farm is at standstill, forecasts stay accurate. The delay follows the time until the first strong-wind hour (above 8 m/s) after the onset: in seeds 0, 2, 3, 4, 5, 6 and 9 the first alarm came within about seven hours of that hour. The longest delays (47 h and 68 h, seeds 3 and 7) belong to streams in which a calm spell of two to three days followed the onset, so there was nothing to detect. Every alarm in the drift scenario came after the true onset: the number of false alarms before the onset was zero in all ten seeds.

Table 6.4 Detection delay and early retraining activity per seed

Seed	Delay (h)	Retrainings in first 3 weeks	Later retrainings	False alarms before onset	MAE reduction vs static (%)
0	16	6	3	0	75.4
1	25	4	20	0	75.7
2	2	8	17	0	76.4
3	47	4	5	0	75.9
4	7	6	3	0	72.6
5	19	6	14	0	75.2
6	9	6	4	0	76.4
7	68	7	8	0	75.6
8	15	9	13	0	73.1
9	7	6	13	0	73.2
Mean	**21.5**	**6.2**	**10.0**	**0**	**75.0**

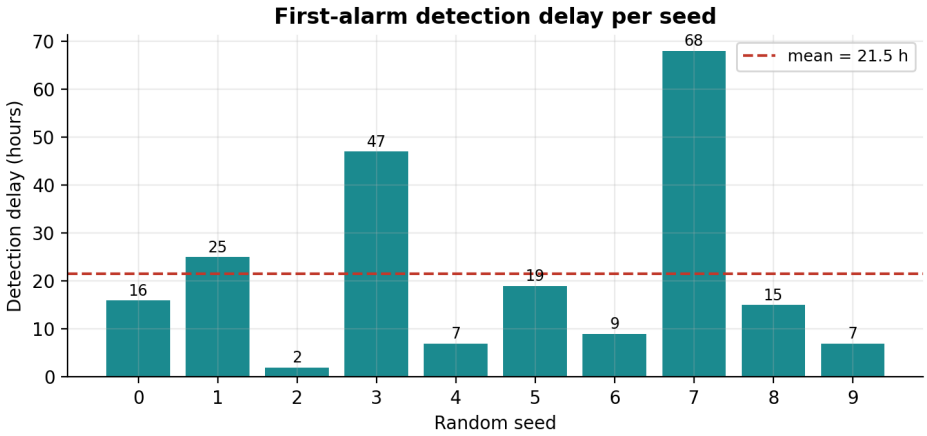


Figure 6.10 First-alarm detection delay for each random seed

6.4.2 Monitoring signals

Figure 6.11 shows the monitored signals for seed 0. In the upper panel, the standardised error fluctuates around one before the onset, jumps at the onset and returns to one after the retrainings. In the lower panel, the Page-Hinkley statistic stays close to zero for 2 weeks, crosses the threshold λ within a day of the onset and triggers the first retraining, then triggers again whenever the remaining error is still accumulating.

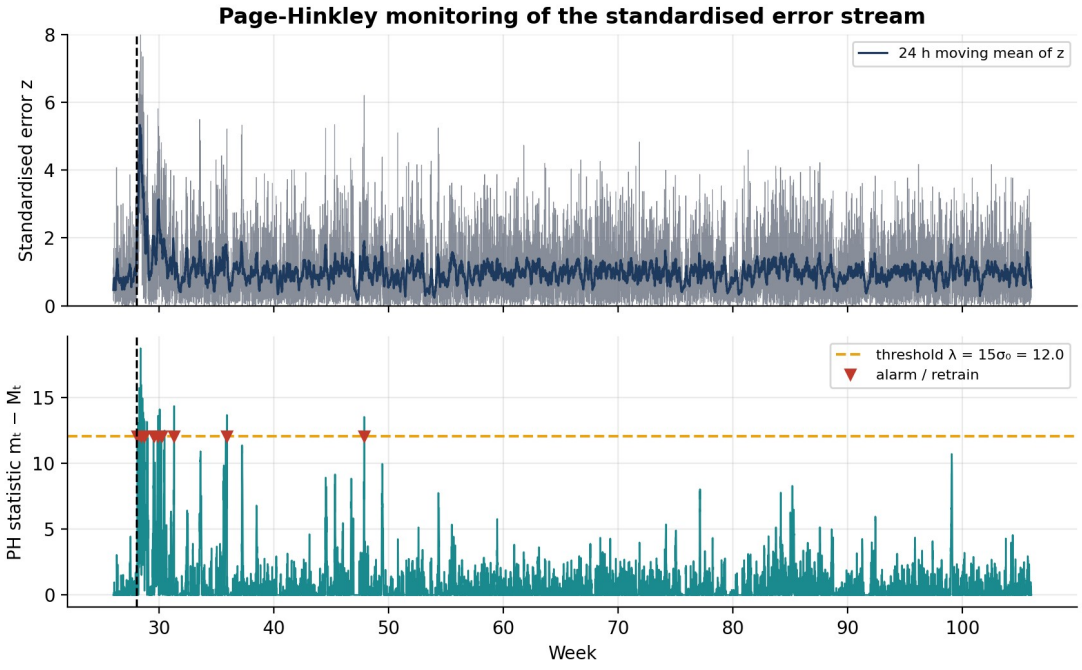


Figure 6.11 Standardised error and Page-Hinkley statistic over the stream (seed 0)

Of the 162 alarms raised across the ten drift runs, 161 came from the Page-Hinkley error detector and 1 from the input-monitor channel. The input monitors were therefore of marginal use in this experiment. Figure 6.12 shows why. The KS p-value and the PSI of wind speed repeatedly cross their thresholds throughout the stream, with no visible response specific to the onset: the natural week-to-week variability of an autocorrelated wind series is larger than the 4% anemometer-scale change. If the input monitors were

allowed to trigger retraining on their own, they would cause constant false alarms; requiring corroboration from the error stream is what keeps the no-drift false-alarm count at zero.

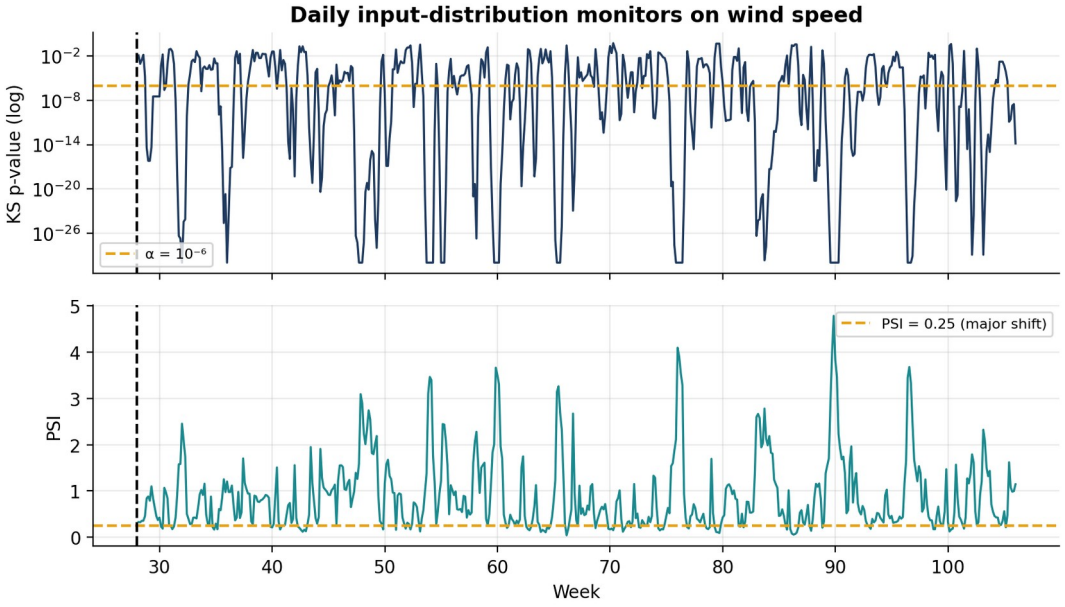


Figure 6.12 Daily KS p-value and PSI of wind speed (seed 0, drift scenario)

6.4.3 Retraining activity

Figure 6.13 shows when the adaptive model retrained. In the drift scenario, 6.2 retrainings on average occur in the first three weeks after the onset, which is the period in which the controller is converging to the new regime (each retraining adds more post-change data to the training window). A further 10.0 retrainings on average are spread over the following 77 weeks (range 3 to 20 per seed). These later alarms are not counted as false alarms because the stream is no longer the same as before the onset, but they show that the detector sometimes reacts to bursts of large error in the post-drift regime, for example an unusually long high-wind spell in the wake sector. In the no-drift scenario there are no retrainings at all.

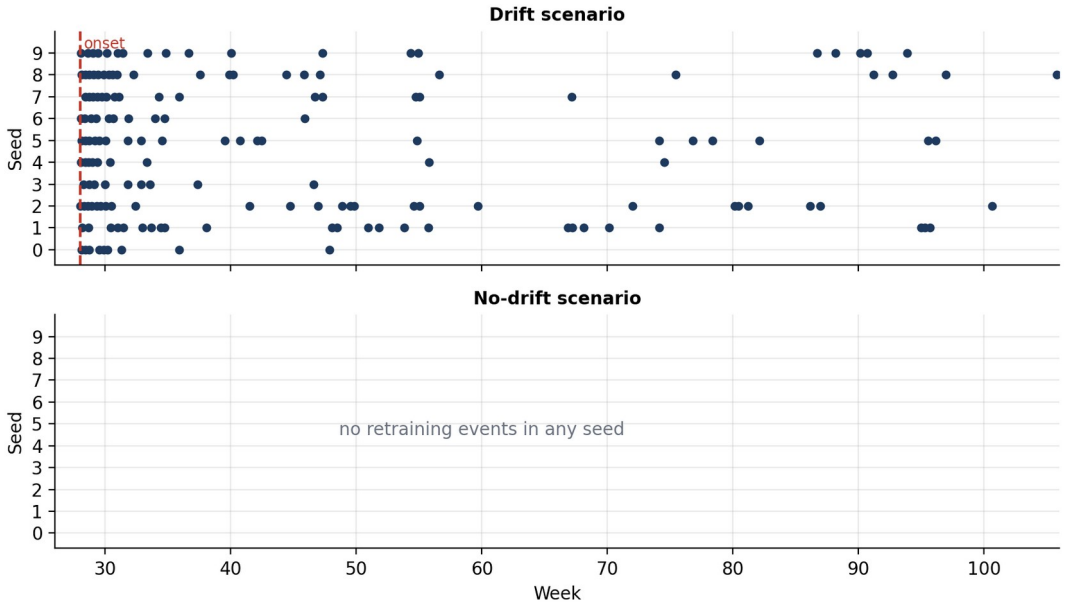


Figure 6.13 Retraining events of the adaptive model for all ten seeds in both scenarios

6.5 Computational Cost

The adaptive model retrained 16.2 ± 6.6 times per run against 80 times for the weekly schedule, so 4.9 times fewer. The cumulative retraining time was 1.98 ± 0.82 s against 9.59 ± 0.53 s, and one retraining took about 0.12 s in both policies. The monitoring itself (error standardisation, Page-Hinkley update and the daily KS and PSI checks) costs about 0.049 ms per hourly step, which is negligible. In the no-drift scenario the adaptive model costs nothing but the monitoring, while the weekly schedule spends 9.92 ± 0.39 s on retrainings that make the forecasts worse.

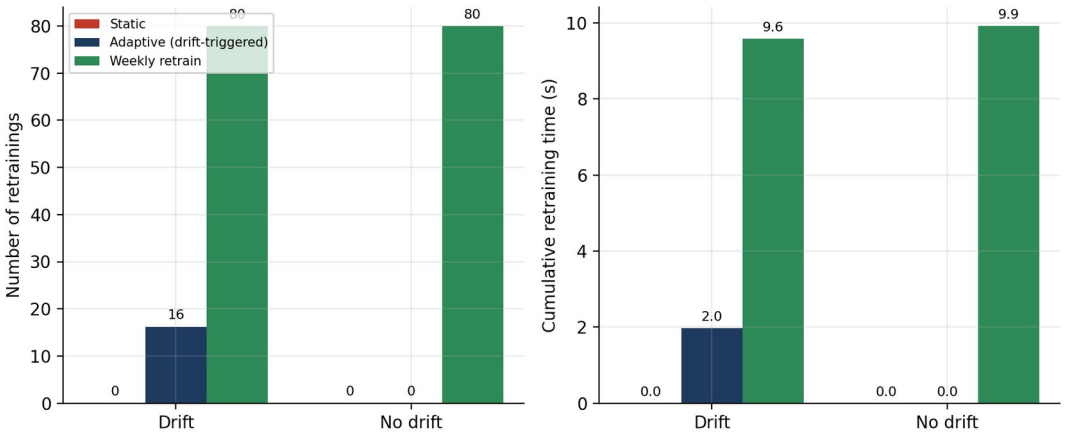


Figure 6.14 Number of retrainings and cumulative retraining time

Table 6.5 Computational cost per run (mean over 10 seeds)

Quantity	Adaptive	Weekly	Ratio
Retrainings (drift scenario)	16.2	80	4.9 × fewer
Retraining time, drift (s)	1.98	9.59	4.9 × less
Retrainings (no-drift scenario)	0	80	–

Quantity	Adaptive	Weekly	Ratio
Retraining time, no drift (s)	0.00	9.92	–
Monitoring cost per hour (ms)	0.049	0	–

Wall-clock time on a single CPU core. Absolute times depend on hardware; ratios are the meaningful quantity.

6.6 Statistical Significance

Paired tests over the ten seeds (Table 6.6) confirm the headline result. The static-versus-adaptive MAE difference of 101.0 kW has a paired t-test of t = 63.5 (p = 3.0e-13) and a Wilcoxon p-value of 0.0020; with ten pairs the smallest attainable two-sided Wilcoxon p-value is 0.002, which is reached because all ten differences have the same sign. The adaptive-versus-weekly difference is small, 1.5 kW in MAE, but still significant at the 5% level (t-test p = 0.014, Wilcoxon p = 0.027), and clearly so in RMSE (p = 2.2e-4). The advantage over the weekly schedule is real but modest, whereas the advantage over the static model is large.

Table 6.6 Paired comparison of MAE over ten seeds (drift scenario)

Comparison	Mean MAE difference (kW)	t-test p	Wilcoxon p	Significant at 5%
Static − Adaptive	101.0	3.0e-13	0.0020	Yes
Weekly − Adaptive	1.5	0.014	0.027	Yes
Static − Weekly	99.4	2.0e-13	0.0020	Yes

6.7 Comparison of Drift Detectors

The Page-Hinkley detector was compared with three alternatives in the same pipeline: a one-sided CUSUM on the standardised error (k = 0.5, h = 8 in σ_0 units), ADWIN from the River library [24] on the same error stream (δ = 0.002), and a KS + PSI input-only trigger with no error corroboration. CUSUM and ADWIN were run with common default settings and were not tuned for this problem, so the comparison indicates robustness under default settings, not the best that each method could do. Five seeds were used per detector.

Table 6.7 Comparison of drift detectors (mean over 5 seeds)

Detector	Drift: MAE (kW)	Retrain-ings	Delay (h)	Detected	No drift: alarms per run	No drift: MAE (kW)
Page-Hinkley (proposed)	**33.7**	**15.2**	**19.4**	**5/5**	**0.0**	**25.7**
CUSUM	33.8	37.4	12.6	5/5	7.8	26.8
ADWIN	36.2	9.2	40.6	5/5	1.6	27.4
KS + PSI only	35.0	153.4	128.6	5/5	153.0	32.4

Static no-drift MAE for these seeds: 25.7 kW. Delay is the mean over detected runs.

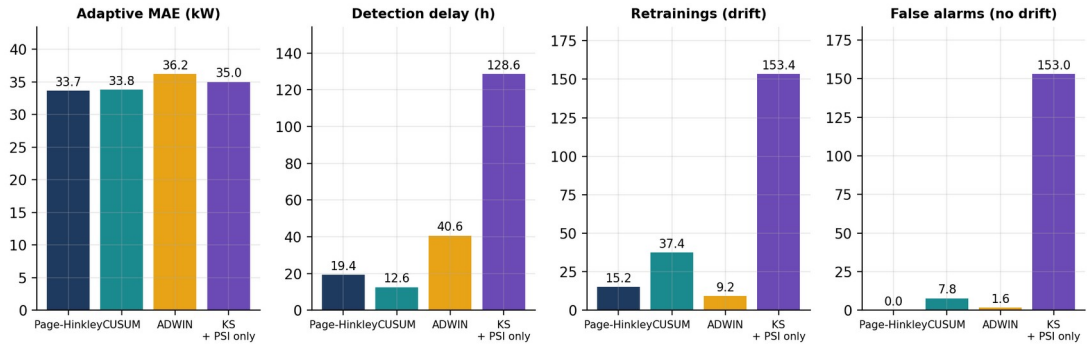


Figure 6.15 Comparison of drift detectors on accuracy, delay, retraining count and false alarms

All error-based detectors recover the accuracy lost to drift (MAE between 33.7 and 36.2 kW against 135.9 kW for the static model), but they differ greatly in what they cost. **CUSUM** reacts fastest (12.6 h) but raises 7.8 alarms per run even when nothing changes and retrains 37.4 times in the drift scenario, which also worsens the no-drift MAE to 26.8 kW. **ADWIN** is conservative: 9.2 retrainings but a delay of 40.6 h, a slightly higher drift MAE (36.2 kW) and 1.6 false alarms per no-drift run, probably because its Hoeffding bound is sensitive to the heavy-tailed standardised errors. The **KS + PSI-only** trigger is unusable on its own: it fires on the natural variability of autocorrelated wind, raises 153 alarms per no-drift run (limited only by the 48-hour cool-down), has a mean delay of 129 h, and raises the no-drift MAE to 32.4 kW (26% worse than static). The Page-Hinkley detector gave the best balance: no false alarm, 15.2 retrainings and a delay of 19.4 h.

6.8 Sensitivity to the Detection Threshold

The threshold λ of the Page-Hinkley test controls the trade-off between speed and false alarms. It was varied from 6σ_0 to 40σ_0 (the baseline is 15σ_0) in both scenarios (Table 6.8, Figure 6.16).

Table 6.8 Effect of the Page-Hinkley threshold λ (mean over 5 seeds)

λ (× σ_0)	Drift: delay (h)	Drift: retrain-ings	Drift: MAE (kW)	No drift: alarms per run	No drift: MAE (kW)
6	12.4	45.8	33.6	18.6	27.8
10	14.0	25.0	34.0	3.8	26.4
15	**19.4**	**15.2**	**33.7**	**0.0**	**25.7**
25	24.4	9.4	34.5	0.0	25.7
40	30.4	8.8	37.7	0.0	25.7

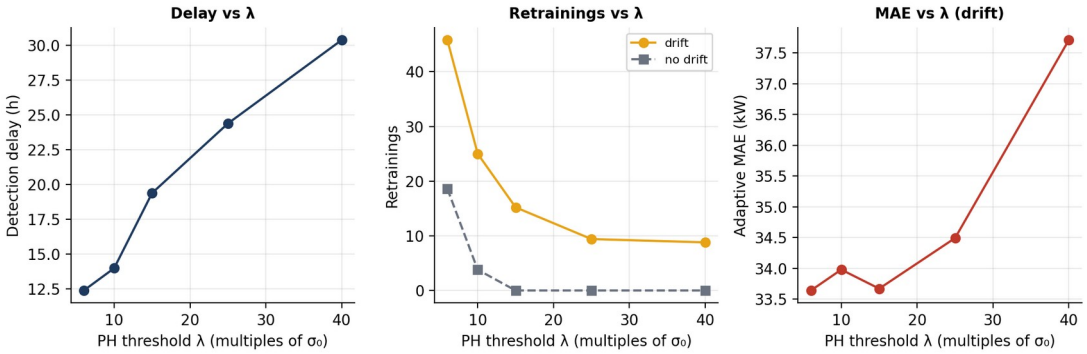


Figure 6.16 Effect of the threshold λ on delay, retraining count and accuracy

A low threshold (λ = 6σ_0) shortens the delay to 12.4 h but at the price of 18.6 false alarms per no-drift run and 45.8 retrainings in the drift scenario. At λ = 10σ_0 a few false alarms still occur (3.8 per run). From λ = 15σ_0 upwards no false alarm occurred. Raising the threshold further reduces retraining (9.4 at 25σ_0) with almost no loss of accuracy (34.5 kW), but at 40σ_0 the delay grows to 30.4 h and the MAE rises to 37.7 kW. The usable range is therefore about 15σ_0 to 25σ_0; the baseline of 15σ_0 is at the sensitive end of that range.

6.9 Sensitivity to Cool-Down and Retraining Window

The cool-down does not change the first detection (the delay is 19.4 h for all settings) but controls how often the model is retrained afterwards (Table 6.9, Figure 6.17). Lengthening it from 12 to 96 hours reduces the retrainings from 22.0 to 11.8 per run without raising the MAE (33.0 to 33.5 kW), and only at 168 hours does the MAE rise slightly (34.3 kW). The 48-hour baseline is a reasonable but not optimal compromise.

Table 6.9 Effect of the cool-down period (drift scenario, 5 seeds)

Cool-down (h)	Retrainings	MAE (kW)	Retraining time (s)
12	22.0	33.0	3.49
24	20.4	33.3	3.56
48	**15.2**	**33.7**	**2.60**
96	11.8	33.5	1.95
168	10.4	34.3	1.71

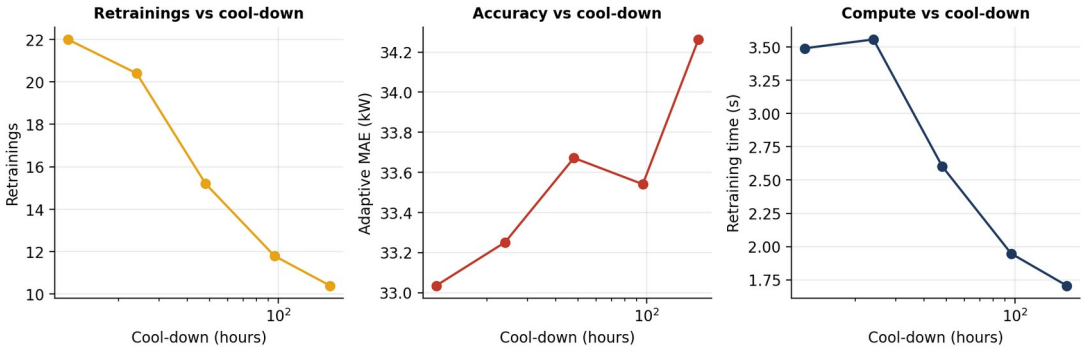


Figure 6.17 Effect of the cool-down period on retraining count, accuracy and compute

The retraining window has a clearer effect (Table 6.10, Figure 6.18). A two-week window gives a clearly higher error (38.3 kW) and more retrainings (31.2), because too few examples are available for the boosted-tree model to learn the whole power curve. Four to eight weeks perform alike (32.9 to 33.7 kW); the six-week window was marginally the best.

Table 6.10 Effect of the retraining window (drift scenario, 5 seeds)

Window (weeks)	Retrainings	MAE (kW)	Retraining time (s)
2	31.2	38.3	3.71
4	**15.2**	**33.7**	**2.60**
6	12.4	32.9	2.35
8	13.8	33.5	2.57

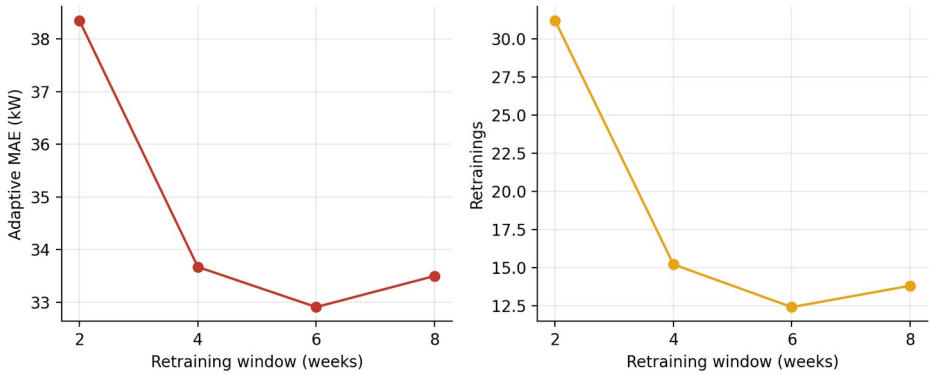


Figure 6.18 Effect of the retraining window length

Note on tuning

The baseline settings ($\lambda = 15\sigma_0$, 48 h cool-down, 4-week window) were fixed before the sensitivity studies and were not changed afterwards, although a 96-hour cool-down or a 6-week window is marginally better. This avoids selecting settings on the same data used for the final comparison.

6.10 Sensitivity to Drift Magnitude

The baseline drift (magnitude 1.0) is large. To learn when adaptation stops paying off, the three drift components were scaled together by a factor between 0 (no drift) and 1.5 (Table 6.11, Figure 6.19).

Table 6.11 Effect of drift magnitude (5 seeds)

Magnitude	Static MAE (kW)	Adaptive MAE (kW)	MAE reduction (%)	Detected	Retrain-ings
0	25.7	25.7	0.0	0/5	0.0
0.1	28.4	28.4	0.0	0/5	0.0
0.25	40.2	34.0	15.4	4/5	5.2
0.5	69.8	32.6	53.3	5/5	9.8
1	**135.9**	**33.7**	**75.2**	**5/5**	**15.2**
1.5	202.0	34.7	82.8	5/5	21.2

Magnitude 0 is the no-drift scenario. The adaptive model equals the static model when no alarm fires.

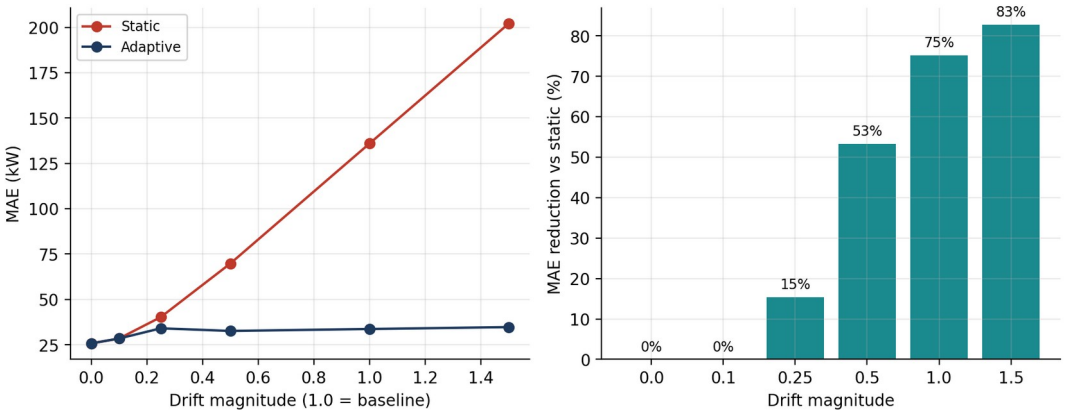


Figure 6.19 Static and adaptive MAE and the relative gain as a function of drift magnitude

The benefit grows with the size of the drift (15% at 0.25, 53% at 0.5, 75% at 1.0, 83% at 1.5), while the adaptive MAE stays between 32.6 and 34.7 kW once the model has adapted. At magnitude 0.1 the drift raised the static MAE only from 25.7 to 28.4 kW and was never detected: the error rise was too small to cross the threshold, and the adaptive model made no change. This is the correct outcome, since retraining would gain little. At magnitude 0.25 the situation is borderline: 4 of 5 runs detected the drift and the detection that did occur came late (mean 2176 h among the detected runs, so those alarms may reflect ordinary error fluctuation as much as the drift), yet the gain is still 15%. Drift whose effect on error is comparable to ordinary variability cannot be detected quickly with an error-based test.

6.11 Verification of the Hypotheses

Table 6.12 Outcome of the research hypotheses

ID	Outcome	Evidence
H1	Supported	MAE 134.7 → 33.7 kW (75.0% lower), p = 3e-13; lower in all 10 seeds
H2	Supported	Zero alarms in 10 no-drift seeds; adaptive forecasts identical to static; no false alarm before the onset in the drift runs
H3	Supported (modest accuracy edge)	MAE 33.7 vs 35.3 kW (p = 0.014) with 4.9× fewer retrainings; in the no-drift scenario weekly retraining is 30.2% worse
H4	Supported	Benefit rises from 0% (magnitude 0.1) to about 83% (magnitude 1.5); detector settings trade delay against false alarms and retraining count

6.12 Practical Guidance: When Does Adaptation Pay Off?

Table 6.13 Guidance derived from the experiments

Situation	Observation	Recommendation
No change in the process	Adaptive model idle; weekly retraining worsens MAE by 30%	Do not retrain on a calendar; monitor only
Abrupt, large change	MAE falls by about 75-83%; delay about one day	Use an error-based sequential detector with cool-down
Small change (≤ 0.1 of baseline)	Not detected; MAE rise small	No action needed; accept the small loss
Change comparable to noise (≈ 0.25)	Detection unreliable and late	Combine with scheduled retraining as a safety net
Natural input variability	KS/PSI fire constantly	Use input monitors only as corroboration
Very low threshold	Fast but many false alarms	Keep λ at 15-25 σ_0

6.13 Limitations and Threats to Validity

- **Synthetic data.** The generator is physically motivated but simpler than a real farm. Absolute error values (kW) should not be transferred to a real site; the relative behaviour (detection, cost, no-drift behaviour) is the contribution.
- **One drift type.** Only an abrupt drift was injected. Gradual, incremental and recurring drift, which are common in practice, may be detected later and need different adaptation.
- **Settings chosen on the generator.** The detector and controller settings were fixed on the same generator that produced the results. The sensitivity studies show that the conclusions hold over a range of settings, but performance on measured data may need re-tuning.

- **Strong lag features.** Power persistence makes the forecaster accurate in the no-drift case; for longer horizons, where lagged power is less informative, errors and drift sensitivity will differ.
- **Late secondary alarms.** About ten retrainings per run occur long after the drift transient. They are not counted as false alarms, but some may be unnecessary; a longer cool-down reduces them (Table 6.9).
- **Untuned alternatives.** CUSUM and ADWIN were run with default settings.
- **Immediate labels.** Actual power is assumed to be available one hour after the forecast. Real systems may have delays or missing data.
- **Sample size.** Ten seeds (five in the sensitivity studies) give stable means, but confidence intervals for some sensitivity entries are wide.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project set out to find out whether concept-drift detection and adaptive retraining can keep wind-power forecasts reliable under changing conditions, and at what cost. A modular framework was designed and implemented: a gradient-boosted forecaster, a standardised error monitor with a Page-Hinkley detector, daily KS and PSI input checks, and an adaptation controller that retrains on a recent, drift-aware window under a cool-down guard. It was evaluated against a static model and a weekly-retrained model in a prequential protocol with an abrupt drift at week 28, a no-drift control scenario and ten random seeds.

The findings are as follows. Under drift, the adaptive model lowered the MAE from 134.7 kW to 33.7 kW (75.0%) and the nRMSE from 8.92% to 2.65%, in every seed and with overwhelming statistical significance. The drift was detected in all runs with a mean delay of 21.5 hours, and the delay mostly reflects the time until the next strong-wind hour, since the drift has no effect on the error while the farm is idle. No false alarm occurred before the onset, and in the no-drift scenario the adaptive model did not retrain at all. Compared with weekly retraining, the adaptive model was slightly but significantly more accurate with 4.9 times fewer retrainings, while weekly retraining made the no-drift forecasts 30.2% worse.

The negative and cautionary results are equally important. Input-distribution tests (KS and PSI) are unreliable on their own for autocorrelated wind data: they fire constantly and, used alone, produced more than a hundred false alarms per no-drift run. They are useful only as corroboration of an error-based alarm. Drift whose effect on error is as small as ordinary variability is either not detected or detected late, but in that case adaptation also brings little. About ten secondary retrainings per run occurred long after the transient, showing that the controller can be tuned further.

Taken together, the results support the view that adaptation should be evidence-driven: a lightweight statistical trigger on a standardised error stream delivers most of the benefit of continual retraining at a fraction of its cost and with no harm when nothing changes. The caveat is that all results are from synthetic data and a single drift type, so they demonstrate the mechanism and its trade-offs; they do not yet establish performance on a particular real wind farm.

7.2 Future Scope

1. **Validation on measured data.** Apply the pipeline to multi-season SCADA or NWP-based farm data from public sources, with known maintenance and curtailment records as reference change points.
2. **More drift types.** Evaluate gradual, incremental and recurring drift, and combinations of virtual and real drift, with change-point estimation suited to slow change.
3. **Multi-step and probabilistic forecasts.** Extend to horizons of several hours to a day and to quantile or density forecasts, monitoring calibration (for example CRPS or coverage) in addition to point error.
4. **Spatial extension.** Monitor several turbines or farms jointly and share information through transfer learning when a new site has little history.
5. **Smarter adaptation.** Replace the full refit by incremental updates, add a change-point-based window length, and learn the cool-down and threshold from the data.
6. **Explainable alarms.** Attribute an alarm to the wind-speed range or direction sector in which the error rose, so that the operator can link it to a physical cause.
7. **Operational integration.** Add a dashboard with live alarm status, delayed or missing label handling, and automatic reports for maintenance planning.

APPENDIX A
SUSTAINABLE DEVELOPMENT GOALS

The proposed drift-adaptive forecasting framework supports several Sustainable Development Goals (SDGs) of the United Nations. By making wind-power forecasts more reliable and cheaper to maintain, it helps wind energy to be integrated into power systems with fewer reserves and less curtailment.

A.1 SDG 7: Affordable and Clean Energy

Reliable forecasts reduce the reserve capacity that grid operators must hold to cover forecast errors, which allows a higher share of wind power at lower system cost. A forecaster that notices when it has become outdated keeps this benefit over the whole life of a wind farm and not only in the months after commissioning.

A.2 SDG 13: Climate Action

Better wind forecasts reduce the need for fast-ramping fossil back-up generation and so lower avoided emissions. The detector also gives early evidence of turbine derating or wake losses, which supports timely maintenance.

A.3 SDG 9: Industry, Innovation and Infrastructure

The project demonstrates a lightweight monitoring and adaptation layer that can be added to existing forecasting systems without a new deep-learning stack, which supports innovation in energy infrastructure.

A.4 SDG 12: Responsible Consumption and Production

Retraining only when evidence demands avoids unnecessary computation. In the experiments the adaptive policy used about five times fewer retrainings than a weekly schedule and none at all in the no-drift scenario.

A.5 SDG 4: Quality Education

The open, reproducible experiment framework serves as a teaching example for time-series forecasting, concept drift and honest model evaluation.

APPENDIX B
SOURCE CODE

This appendix lists the simulation engine (sim.py) used for all experiments and the driver scripts that produce the result files. All numbers in Chapter 6 were produced by this code with random seeds 0 to 9.

B.1 Simulation Engine (sim.py)

```
"""Adaptive wind-power forecasting with concept-drift detection - simulation engine."""
import time
import numpy as np
from scipy import stats
from sklearn.ensemble import HistGradientBoostingRegressor

RATED = 2000.0            # kW
HPW = 168                 # hours per week
TOTAL_WEEKS = 106
INIT_WEEKS = 26
DRIFT_WEEK = 28
WIN = 4 * HPW             # retraining window (672 h)
FEATS = ["v", "sin_d", "cos_d", "temp", "p_l1", "p_l2", "p_l24", "p_m6", "v_l1", "v_m3"]


def power_curve(v, temp, vin=3.0, vr=12.0, vout=25.0):
    rho = 101325.0 / (287.05 * (temp + 273.15))
    corr = rho / 1.225
    p = RATED * (v ** 3 - vin ** 3) / (vr ** 3 - vin ** 3) * corr
    p = np.where(v < vin, 0.0, p)
    p = np.minimum(p, RATED)
    p = np.where(v >= vout, 0.0, p)
    return np.maximum(p, 0.0)


def ar1(rng, n, phi):
    x = np.zeros(n)
    e = rng.standard_normal(n)
    for i in range(1, n):
        x[i] = phi * x[i - 1] + np.sqrt(1 - phi ** 2) * e[i]
    return x


def generate(seed, drift=True, magnitude=1.0, n_weeks=TOTAL_WEEKS,
drift_week=DRIFT_WEEK):
    """Hourly synthetic farm-level data. Returns dict of arrays."""
    rng = np.random.default_rng(seed)
    n = n_weeks * HPW
    t = np.arange(n)
    doy = (t / 24.0) % 365
    season = 1.0 * np.cos(2 * np.pi * (doy - 20) / 365)            # winter windier
    diurnal = 0.7 * np.sin(2 * np.pi * ((t % 24) - 14) / 24)
    v_true = 7.6 + 1.3 * season + diurnal + 2.7 * ar1(rng, n, 0.96)
    v_true = np.clip(v_true, 0.0, 30.0)
    temp = 17 + 9 * np.cos(2 * np.pi * (doy - 200) / 365) + 3 * np.sin(2 * np.pi * ((t %
24) - 15) / 24) \
        + 1.5 * ar1(rng, n, 0.97)
    d = 225 + 55 * ar1(rng, n, 0.97)
    d = d % 360
    p_true = power_curve(v_true, temp)
```

```
    v_meas = v_true + 0.15 * rng.standard_normal(n)
    d_meas = d + 4 * rng.standard_normal(n)
    onset = None
    if drift:
        onset = drift_week * HPW
        m = magnitude
        after = t >= onset
        # (1) real concept drift: rotor/ageing + wake of a new obstacle in SW sector
        deg = 1.0 - 0.075 * m
        sector = ((d > 200) & (d < 265)).astype(float)
        wake = 1.0 - 0.085 * m * sector
        p_true = np.where(after, p_true * deg * wake, p_true)
        # (2) anemometer recalibration (input / virtual drift)
        v_meas = np.where(after, v_meas * (1 + 0.04 * m), v_meas)
    p_obs = p_true * (1 + 0.008 * rng.standard_normal(n)) + 7.0 * rng.standard_normal(n)
    p_obs = np.clip(p_obs, 0.0, RATED)
    return dict(v=v_meas, v_true=v_true, d=d_meas, temp=temp, p=p_obs, p_clean=p_true,
onset=onset, n=n)


def features(D):
    p, v = D["p"], D["v"]
    n = D["n"]
    rad = np.deg2rad(D["d"])
    X = np.zeros((n, len(FEATS)))
    X[:, 0] = v
    X[:, 1] = np.sin(rad)
    X[:, 2] = np.cos(rad)
    X[:, 3] = D["temp"]
    lag = lambda a, k: np.concatenate([np.full(k, a[0]), a[:-k]])
    X[:, 4] = lag(p, 1)
    X[:, 5] = lag(p, 2)
    X[:, 6] = lag(p, 24)
    cs = np.cumsum(np.insert(p, 0, 0))
    m6 = np.array([(cs[i] - cs[max(i - 6, 0)]) / max(min(i, 6), 1) for i in range(n)])
    X[:, 7] = m6
    X[:, 8] = lag(v, 1)
    csv = np.cumsum(np.insert(v, 0, 0))
    X[:, 9] = np.array([(csv[i] - csv[max(i - 3, 0)]) / max(min(i, 3), 1) for i in
range(n)])
    return X


def new_model(seed):
    return HistGradientBoostingRegressor(max_iter=120, learning_rate=0.1, max_depth=6,
                                         min_samples_leaf=20, random_state=seed)


HALF_LIFE = 336.0
PRE_W = 0.08


def fit_recent(seed, X, y, lo, hi, half_life=None, tau=None):
    """Fit on rows lo..hi-1 with exponentially decaying recency weights."""
    hl = HALF_LIFE if half_life is None else half_life
    age = (hi - 1 - np.arange(lo, hi)).astype(float)
    w = 0.5 ** (age / hl)
    if tau is not None:   # drift-aware weighting: down-weight data before the estimated
change point
        w = np.where(np.arange(lo, hi) < tau, w * PRE_W, w)
    m = new_model(seed)
    m.fit(X[lo:hi], y[lo:hi], sample_weight=w)
    return m
```

```
def psi(ref, cur, bins=10):
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf
    r = np.histogram(ref, edges)[0] / len(ref)
    c = np.histogram(cur, edges)[0] / len(cur)
    r, c = np.clip(r, 1e-4, None), np.clip(c, 1e-4, None)
    return float(np.sum((c - r) * np.log(c / r)))


class PageHinkley:
    """Page-Hinkley test for an increase in mean, against a fixed reference level mu0."""
    def __init__(self, mu0, delta, lam):
        self.mu0, self.delta, self.lam = mu0, delta, lam
        self.reset()

    def reset(self):
        self.m = 0.0
        self.M = 0.0
        self.t_min = None
        self.k = 0

    def update(self, x, t):
        self.m += x - self.mu0 - self.delta
        if self.m < self.M:
            self.M = self.m
            self.t_min = t
        return (self.m - self.M) > self.lam


class Cusum:
    def __init__(self, mu0, sigma0, k=0.5, h=8.0):
        self.mu0, self.s0, self.k, self.h = mu0, sigma0, k, h
        self.g = 0.0

    def reset(self):
        self.g = 0.0

    def update(self, x, t):
        z = (x - self.mu0) / self.s0
        self.g = max(0.0, self.g + z - self.k)
        return self.g > self.h


class AdwinDet:
    def __init__(self, delta=0.002):
        from river.drift import ADWIN
        self.delta = delta
        self.a = ADWIN(delta=delta)
        self.prev = None

    def reset(self):
        from river.drift import ADWIN
        self.a = ADWIN(delta=self.delta)

    def update(self, x, t):
        before = self.a.estimation
        self.a.update(x)
        return bool(self.a.drift_detected and self.a.estimation > before)


def run_static(model, X, y, i0):
    pred = model.predict(X[i0:])
    return pred
```

```
def run_weekly(seed, X, y, i0, base_model):
    """Blind weekly retraining on the last WIN hours."""
    n = len(y)
    pred = np.zeros(n - i0)
    model = base_model
    t_retrain = 0.0
    k = 0
    i = i0
    while i < n:
        j = min(i + HPW, n)
        pred[i - i0:j - i0] = model.predict(X[i:j])
        t0 = time.perf_counter()
        m = fit_recent(seed, X, y, max(0, j - WIN), j)
        t_retrain += time.perf_counter() - t0
        model = m
        k += 1
        i = j
    return pred, k, t_retrain


def run_adaptive(seed, X, y, D, i0, base_model, cal, detector="ph", use_input=True,
                 delta_f=0.6, lam_f=15.0, cooldown=48, block=24, ks_alpha=1e-6,
psi_thr=0.25,
                 win=WIN, record=False):
    """Drift-triggered retraining (test-then-train) on a sliding recent window."""
    n = len(y)
    pred = np.zeros(n - i0)
    model = base_model
    mu0, sigma0 = cal['mu0'], cal['s0']
    zbuf = np.zeros(n - i0)
    if detector == "ph":
        det = PageHinkley(mu0, delta_f * sigma0, lam_f * sigma0)
    elif detector == "cusum":
        det = Cusum(mu0, sigma0)
    elif detector == "adwin":
        det = AdwinDet()
    elif detector in ("none", "kspsi"):
        det = None
    alarms, retrain_t = [], []
    sources = []
    t_retrain = 0.0
    t_detect = 0.0
    last_retrain = -10 ** 9
    trace = {"ph": [], "ks": [], "psi": [], "err": []} if record else None
    i = i0
    while i < n:
        j = min(i + block, n)
        P = model.predict(X[i:j])
        fired = None
        for k in range(i, j):
            e = abs(P[k - i] - y[k]) / cal['means'][np.digitize(P[k - i], cal['edges'])]
            pred[k - i0] = P[k - i]
            zbuf[k - i0] = e
            td = time.perf_counter()
            hit = False
            src = None
            if det is not None and detector != "kspsi":
                if det.update(e, k):
                    hit, src = True, detector
            if record:
                trace["err"].append(e)
```

```
                trace["ph"].append(getattr(det, "m", 0.0) - getattr(det, "M", 0.0) if
detector == "ph" else 0.0)
            # input monitoring once a day
            if (use_input or detector == "kspsi") and k % 24 == 23 and k - i0 > 2 * HPW:
                cur = D["v"][k - HPW + 1:k + 1]
                ref = D["v"][k - HPW - 4 * HPW + 1:k - HPW + 1]
                ks = stats.ks_2samp(ref, cur)
                pv = psi(ref, cur)
                # direction consistency as extra input
                if record:
                    trace["ks"].append((k, ks.pvalue, ks.statistic))
                    trace["psi"].append((k, pv))
                if (ks.pvalue < ks_alpha and pv > psi_thr):
                    # input shift is only an alarm when accompanied by degraded accuracy
                    recent = np.mean(zbuf[max(0, k - i0 - 23):k - i0 + 1])
                    if detector == "kspsi" or recent > mu0 + 2 * sigma0:
                        if not hit:
                            hit, src = True, "input"
            t_detect += time.perf_counter() - td
            if hit and (k - last_retrain) >= cooldown:
                fired = (k, src)
                break
            elif hit:
                if det is not None and hasattr(det, "reset") and detector != "kspsi":
                    det.reset()
        if fired is None:
            i = j
            continue
        k, src = fired
        tau = getattr(det, "t_min", None)
        if tau is None or src == "input":
            tau = k - 24
        tau = max(tau - 12, 0)
        alarms.append(k)
        sources.append(src)
        t0 = time.perf_counter()
        lo = max(0, k + 1 - win)
        m = fit_recent(seed, X, y, lo, k + 1, tau=tau)
        t_retrain += time.perf_counter() - t0
        model = m
        retrain_t.append(k)
        last_retrain = k
        if det is not None and hasattr(det, "reset"):
            det.reset()
        # prediction for hours after k within this block are re-done with new model
        i = k + 1
    res = dict(pred=pred, alarms=alarms, sources=sources, n_retrain=len(retrain_t),
               t_retrain=t_retrain, t_detect=t_detect, retrain_t=retrain_t)
    if record:
        res["trace"] = trace
    return res


def metrics(y, p):
    e = p - y
    mae = float(np.mean(np.abs(e)))
    rmse = float(np.sqrt(np.mean(e ** 2)))
    return mae, rmse, rmse / RATED * 100


def calibrate(model_seed, X, y, i0):
    """Rolling-origin validation inside the initial window (origins at weeks 10, 14, 18,
22): models trained
```

```
    like a drift-triggered retrain forecast the next 4 weeks. The pooled residuals give a
heteroscedasticity
    map (mean absolute error per forecast-level bin) so that monitored errors are
standardised:
    z = |e| / m(pred). Returns the initial model and the calibration dict."""
    P, E = [], []
    for wk in (10, 14, 18, 22):
        cut = wk * HPW
        m = fit_recent(model_seed, X, y, max(0, cut - WIN), cut)
        p = m.predict(X[cut:cut + 4 * HPW])
        P.append(p)
        E.append(np.abs(p - y[cut:cut + 4 * HPW]))
    P, E = np.concatenate(P), np.concatenate(E)
    edges = np.quantile(P, np.linspace(0, 1, 7))[1:-1]
    b = np.digitize(P, edges)
    means = np.array([max(E[b == k].mean(), 1.0) for k in range(6)])
    z = E / means[b]
    final = new_model(model_seed)
    final.fit(X[:i0], y[:i0])
    return final, dict(edges=edges, means=means, mu0=float(z.mean()), s0=float(z.std()))


def experiment(seed, drift=True, magnitude=1.0, record=False, detector="ph",
do_weekly=True, **kw):
    D = generate(seed, drift=drift, magnitude=magnitude)
    X = features(D)
    y = D["p"]
    i0 = INIT_WEEKS * HPW
    base, cal = calibrate(seed, X, y, i0)
    yt = y[i0:]
    out = {"D": D, "mu0": cal['mu0'], "s0": cal['s0'], "cal": cal, "i0": i0}
    out["static"] = dict(pred=run_static(base, X, y, i0))
    if do_weekly:
        wp, wk, wt = run_weekly(seed, X, y, i0, base)
    else:
        wp, wk, wt = out["static"]["pred"], 0, 0.0
    out["weekly"] = dict(pred=wp, n_retrain=wk, t_retrain=wt)
    ad = run_adaptive(seed, X, y, D, i0, base, cal, detector=detector, record=record,
**kw)
    out["adaptive"] = ad
    out["y"] = yt
    out["X"] = X
    out["base"] = base
    for k in ("static", "weekly", "adaptive"):
        out[k]["metrics"] = metrics(yt, out[k]["pred"])
    if drift:
        onset = D["onset"]
        first = [a for a in ad["alarms"] if a >= onset]
        out["delay"] = (first[0] - onset) if first else None
        out["false_alarms"] = len([a for a in ad["alarms"] if a < onset])
    else:
        out["delay"] = None
        out["false_alarms"] = len(ad["alarms"])
    return out
```

B.2 Experiment Driver for the Main Results

```
import pickle, numpy as np, time, sys
from sim import *
def slim(o, keep_arrays=False):
    r = {k: o[k] for k in ('mu0','s0','delay','false_alarms')}
    for k in ('static','weekly','adaptive'):
```

```
        r[k] = {kk:vv for kk,vv in o[k].items() if kk not in ('pred','trace')}
    return r
main = {}
t0=time.time()
for drift in (True, False):
    for seed in range(10):
        o = experiment(seed, drift=drift, record=(seed==0))
        r = slim(o)
        if seed == 0:   # keep arrays for figures
            r['arrays'] = dict(y=o['y'], p_static=o['static']['pred'],
p_adapt=o['adaptive']['pred'],
                p_weekly=o['weekly']['pred'], v=o['D']['v'], v_true=o['D']['v_true'],
d=o['D']['d'], temp=o['D']['temp'],
                p=o['D']['p'], p_clean=o['D']['p_clean'], onset=o['D']['onset'],
trace=o['adaptive']['trace'],
                alarms=o['adaptive']['alarms'], sources=o['adaptive']['sources'],
cal=o['cal'], i0=o['i0'])
            # feature importance via permutation on a stream slice
        main[(drift, seed)] = r
        print('main', drift, seed, round(time.time()-t0), flush=True)
pickle.dump(main, open('main.pkl','wb'))

sens = {}
# detector comparison (drift scenario, 5 seeds)
for det in ('ph','cusum','adwin','kspsi'):
    for seed in range(5):
        for drift in (True, False):
            o = experiment(seed, drift=drift, detector=det, do_weekly=False,
use_input=(det!='adwin'))
            sens[('det',det,drift,seed)] = slim(o)
    print('det', det, round(time.time()-t0), flush=True)
# lambda sweep
for lam in (6, 10, 15, 25, 40):
    for seed in range(5):
        for drift in (True, False):
            o = experiment(seed, drift=drift, do_weekly=False, lam_f=float(lam))
            sens[('lam',lam,drift,seed)] = slim(o)
    print('lam', lam, round(time.time()-t0), flush=True)
# cooldown sweep
for cd in (12, 24, 48, 96, 168):
    for seed in range(5):
        o = experiment(seed, drift=True, do_weekly=False, cooldown=cd)
        sens[('cd',cd,True,seed)] = slim(o)
    print('cd', cd, round(time.time()-t0), flush=True)
# window sweep
for w in (2, 4, 6, 8):
    for seed in range(5):
        o = experiment(seed, drift=True, do_weekly=False, win=w*HPW)
        sens[('win',w,True,seed)] = slim(o)
    print('win', w, round(time.time()-t0), flush=True)
# magnitude sweep
for m in (0.0, 0.1, 0.25, 0.5, 1.0, 1.5):
    for seed in range(5):
        o = experiment(seed, drift=(m>0), magnitude=max(m,1e-9), do_weekly=False)
        sens[('mag',m,True,seed)] = slim(o)
    print('mag', m, round(time.time()-t0), flush=True)
pickle.dump(sens, open('sens.pkl','wb'))
print('DONE')
```

B.3 Experiment Driver for the Sensitivity Studies

```
import pickle, time
```

```
from sim import *
def slim(o):
    r = {k: o[k] for k in ('mu0','s0','delay','false_alarms')}
    for k in ('static','weekly','adaptive'):
        r[k] = {kk:vv for kk,vv in o[k].items() if kk not in ('pred','trace')}
    return r
sens = {}; t0 = time.time()
def run(key, **kw):
    sens[key] = slim(experiment(kw.pop('seed'), do_weekly=False, **kw))
for det in ('ph','cusum','adwin','kspsi'):
    for seed in range(5):
        for drift in (True, False):
            run(('det',det,drift,seed), seed=seed, drift=drift, detector=det,
use_input=(det!='adwin'))
            if det == 'ph':   # default configuration doubles as lam=15
                sens[('lam',15,drift,seed)] = sens[('det','ph',drift,seed)]
                if drift:
                    sens[('cd',48,True,seed)] = sens[('det','ph',True,seed)]
                    sens[('win',4,True,seed)] = sens[('det','ph',True,seed)]
                    sens[('mag',1.0,True,seed)] = sens[('det','ph',True,seed)]
    print('det', det, round(time.time()-t0), flush=True)
    pickle.dump(sens, open('sens.pkl','wb'))
for lam in (6, 10, 25, 40):
    for seed in range(5):
        for drift in (True, False):
            run(('lam',lam,drift,seed), seed=seed, drift=drift, lam_f=float(lam))
    print('lam', lam, round(time.time()-t0), flush=True)
for cd in (12, 24, 96, 168):
    for seed in range(5):
        run(('cd',cd,True,seed), seed=seed, drift=True, cooldown=cd)
    print('cd', cd, round(time.time()-t0), flush=True)
for w in (2, 6, 8):
    for seed in range(5):
        run(('win',w,True,seed), seed=seed, drift=True, win=w*HPW)
    print('win', w, round(time.time()-t0), flush=True)
for m in (0.0, 0.1, 0.25, 0.5, 1.5):
    for seed in range(5):
        run(('mag',m,True,seed), seed=seed, drift=(m>0), magnitude=max(m,1e-9))
    print('mag', m, round(time.time()-t0), flush=True)
pickle.dump(sens, open('sens.pkl','wb'))
print('DONE')
```

APPENDIX C
OUTPUT SCREENSHOTS

The figures in this appendix are renderings of the console output produced by the experiment scripts. They show the values generated in the run for seed 0 and the aggregate over ten seeds.

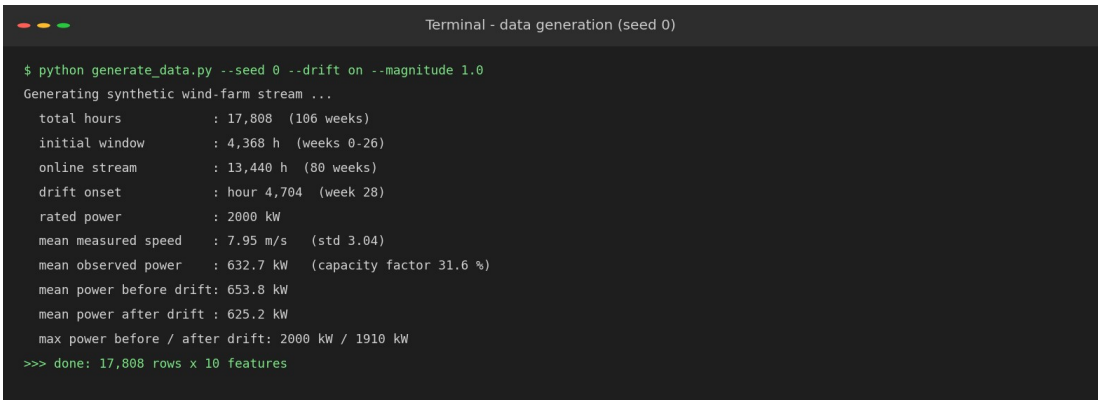


Figure C.1 Console output: synthetic data generation (seed 0)

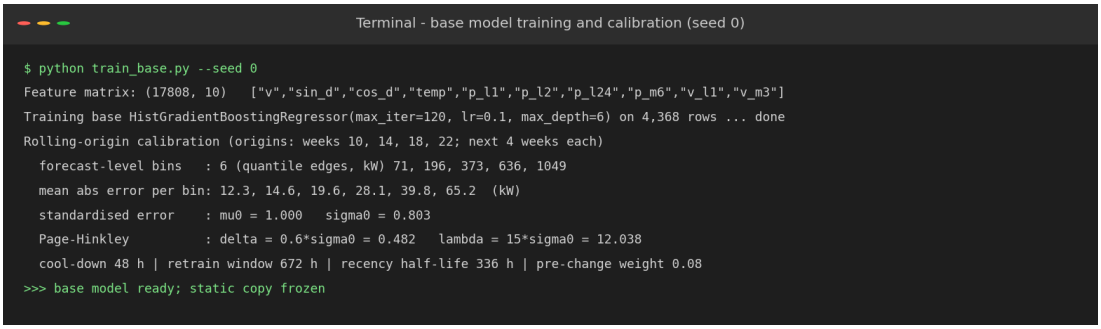


Figure C.2 Console output: base model training and calibration (seed 0)

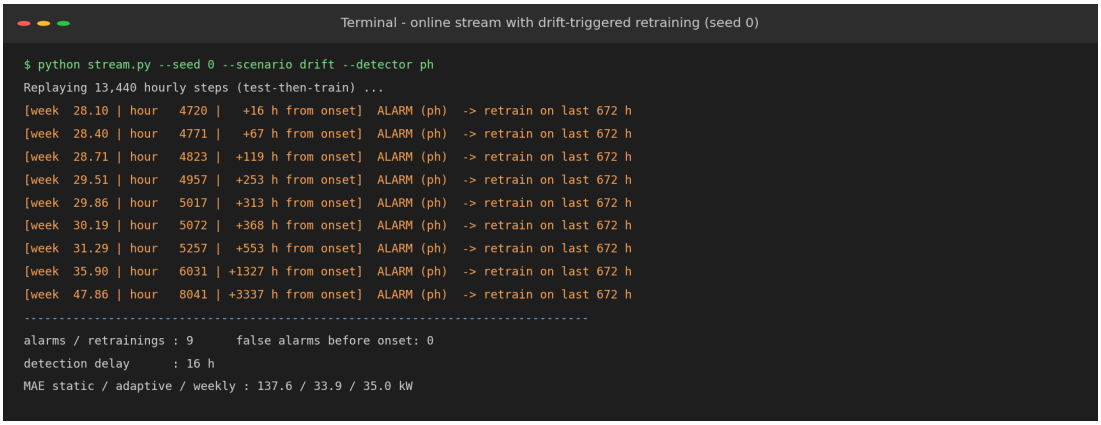


Figure C.3 Console output: online stream with drift-triggered retraining (seed 0)

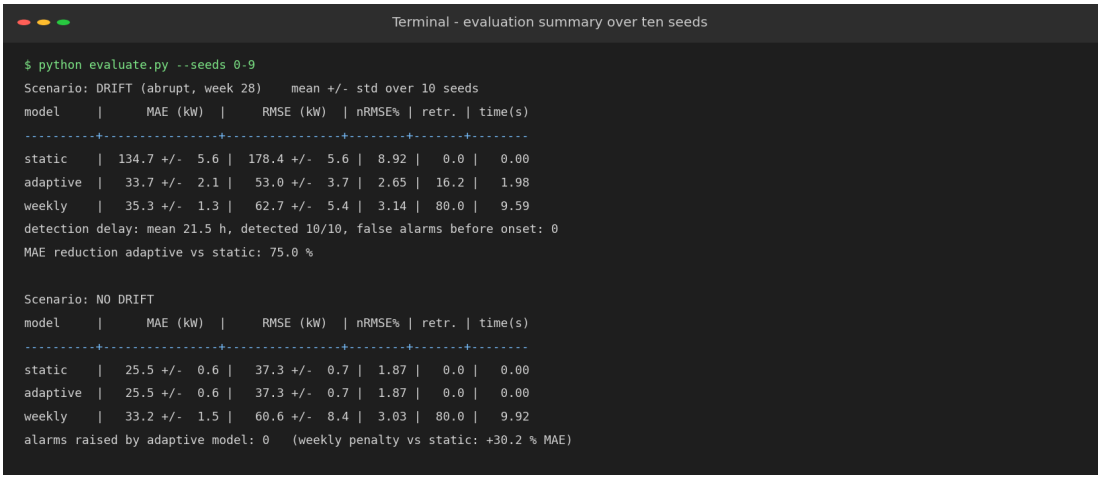


Figure C.4 Console output: evaluation summary over ten seeds

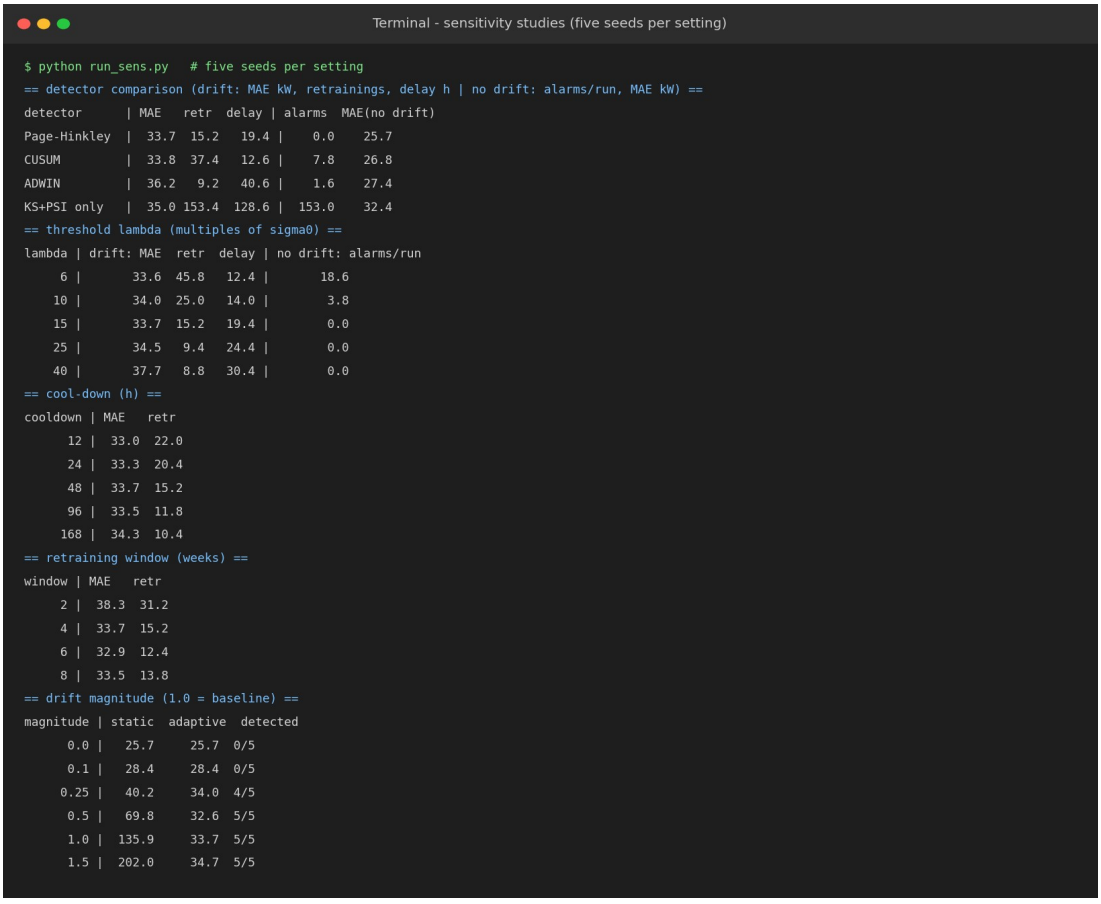


Figure C.5 Console output: sensitivity summary (five seeds)

APPENDIX D
PER-SEED RESULTS

Table D.1 and Table D.2 give the accuracy of every individual run (all values in kW). They allow the reader to verify the means and standard deviations of Chapter 6.

Table D.1 Per-seed accuracy in the drift scenario (kW)

Seed	MAE static	MAE adaptive	MAE weekly	RMSE static	RMSE adaptive	RMSE weekly
0	137.6	33.9	35.0	182.0	52.8	60.8
1	125.9	30.5	34.4	168.3	48.6	60.0
2	144.1	33.9	36.4	185.2	51.3	58.7
3	134.9	32.5	35.1	179.6	50.8	61.2
4	137.0	37.5	36.9	179.6	58.5	66.7
5	141.7	35.1	37.2	186.2	52.4	73.4
6	131.9	31.1	32.9	177.7	48.2	55.0
7	132.9	32.5	35.6	176.9	52.3	64.1
8	128.9	34.7	33.9	171.4	58.9	59.4
9	131.6	35.3	35.1	177.1	55.8	68.2
Mean	**134.7**	**33.7**	**35.3**	**178.4**	**53.0**	**62.7**

Table D.2 Per-seed accuracy in the no-drift scenario (kW)

Seed	MAE static	MAE adaptive	MAE weekly	RMSE static	RMSE adaptive	RMSE weekly
0	25.5	25.5	31.1	37.6	37.6	52.6
1	25.0	25.0	33.9	36.8	36.8	60.3
2	26.3	26.3	33.3	38.0	38.0	51.3
3	25.7	25.7	33.0	37.8	37.8	56.0
4	26.1	26.1	34.6	38.0	38.0	64.9
5	26.3	26.3	35.5	38.1	38.1	74.5
6	25.1	25.1	31.0	36.8	36.8	50.9
7	25.1	25.1	34.2	36.9	36.9	64.4
8	25.0	25.0	31.8	36.3	36.3	58.7
9	25.1	25.1	34.1	36.8	36.8	72.6
Mean	**25.5**	**25.5**	**33.2**	**37.3**	**37.3**	**60.6**

D.1 Configuration Summary

Table D.3 Final configuration of the adaptive system

Component	Setting
Forecaster	HistGradientBoostingRegressor, 120 iterations, learning rate 0.1, max depth 6, min leaf 20
Features	10 (speed, sin/cos direction, temperature, 3 power lags, rolling power, speed lag, rolling speed)
Monitored score	Absolute error divided by the calibrated mean error of the forecast-level bin (6 bins)
Page-Hinkley	Reference μ_0 from calibration; tolerance $\delta = 0.6\sigma_0$; threshold $\lambda = 15\sigma_0$
Input monitors	Daily KS ($p < 1e\text{-}6$) and PSI (> 0.25) on the last 168 h versus the preceding four weeks; accepted only with 24 h mean $z > \mu_0 + 2\sigma_0$
Cool-down	48 hours
Retraining window	672 hours, recency half-life 336 hours, pre-change weight 0.08
Calibration	Rolling origins at weeks 10, 14, 18, 22 with 4-week forecasts

REFERENCES

[1] Y. He, N. Yu, and B. Wang, "Online probability density prediction of wind power considering virtual and real concept drift detection," Appl. Energy, vol. 396, Art. no. 126318, 2025, doi: 10.1016/j.apenergy.2025.126318.

[2] C. Cao, Y. He, Y. Zhou, and S. Wang, "An online probabilistic combination framework for power load forecasting under concept-drifting scenarios," Appl. Energy, vol. 399, Art. no. 126518, 2025, doi: 10.1016/j.apenergy.2025.126518.

[3] C. Cao, Y. He, and Y. Zhou, "Probabilistic multi-step prediction of wind power based on online feature extraction and parallel output network structure," Eng. Appl. Artif. Intell., 2026.

[4] A. Bahi, A. Ourici, I. Gasmi, A. Derrablia, W. Deghmane, and M. A. Ferrag, "FreeGNN: Continual source-free graph neural network adaptation for renewable energy forecasting," arXiv:2603.01657, 2026.

[5] J. Hou, Z. Yan, and Y. Meng, "A hybrid physics and data-driven framework for ultra-short-term wind power forecasting with spatiotemporal learning," J. Renew. Sustain. Energy, vol. 17, no. 3, 2025.

[6] H. Jin, G. Yang, S. Dong, S. Fan, H. Jin, and B. Wang, "Wind power forecasting for newly built wind farms based on deep learning with dual-stage attention mechanism and adaptive transfer learning," Energy, vol. 335, 2025.

[7] M. Yang, Y. Guo, T. Huang, and W. Zhang, "Power prediction considering NWP wind speed error tolerability: A strategy to improve the accuracy of short-term wind power prediction under wind speed offset scenarios," Appl. Energy, vol. 377, 2025.

[8] Y. Yang, Z. Liu, and Z. Yu, "SA-STGCN: A spectral-attentive spatio-temporal graph convolutional network for wind power forecasting with wavelet-enhanced multi-scale learning," Energies, vol. 18, no. 19, Art. no. 5315, 2025.

[9] Y. Wang, F. Zhang, H. Kou, R. Zou, Q. Hu, J. Wang, and D. Srinivasan, "A review of predictive uncertainty modeling techniques and evaluation metrics in probabilistic wind speed and wind power forecasting," Appl. Energy, vol. 396, 2025.

[10] M. Yang, Y. Huang, C. Xu, C. Liu, and B. Dai, "Review of several key processes in wind power forecasting: Mathematical formulations, scientific problems, and logical relations," Appl. Energy, vol. 377, 2025.

[11] Y. Zhong, W. Zhou, and L. Tao, "Source-free time series domain adaptation with wavelet-based multi-scale temporal imputation," Neural Netw., vol. 188, Art. no. 107428, 2025.

[12] M. T. Furqon, M. Pratama, A. Shiddiqi, L. Liu, H. Habibullah, and K. Dogancay, "Time and frequency synergy for source-free time-series domain adaptations," Inf. Sci., vol. 695, Art. no. 121734, 2025.

[13] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACM Comput. Surv., vol. 46, no. 4, pp. 1–37, 2014.

[14] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," IEEE Trans. Knowl. Data Eng., vol. 31, no. 12, pp. 2346–2363, 2019.

[15] E. S. Page, "Continuous inspection schemes," Biometrika, vol. 41, no. 1–2, pp. 100–115, 1954.

[16] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing,” in Proc. SIAM Int. Conf. Data Mining (SDM), 2007, pp. 443–448.

[17] T. Burton, N. Jenkins, D. Sharpe, and E. Bossanyi, Wind Energy Handbook, 2nd ed. Chichester, U.K.: Wiley, 2011.

[18] F. J. Massey Jr., “The Kolmogorov-Smirnov test for goodness of fit,” J. Amer. Statist. Assoc., vol. 46, no. 253, pp. 68–78, 1951.

[19] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[20] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017.

[21] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[22] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” Int. J. Forecasting, vol. 22, no. 4, pp. 679–688, 2006.

[23] J. Gama, R. Sebastião, and P. P. Rodrigues, “On evaluating stream learning algorithms,” Mach. Learn., vol. 90, no. 3, pp. 317–346, 2013.

[24] J. Montiel et al., “River: Machine learning for streaming data in Python,” J. Mach. Learn. Res., vol. 22, no. 110, pp. 1–8, 2021.

[25] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

[26] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” Nature Methods, vol. 17, pp. 261–272, 2020.

[27] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007.